PROFILE



# 염동섭

**게임 클라이언트 프로그래머**

C++ · Unreal Engine 5.4

CONTACT

dsyeom931008@gmail.com

010-7181-7394

GitHub 프로필 ↗

GAME CLIENT DEVELOPMENT



# 게임 클라이언트 개발 포트폴리오

C++ · Unreal Engine 5.4 · DirectX11

01 **Project Stellar** 03–24
근접 액션 전투·AI 시스템 구현과 문제 해결

02 **Asset Reference Inspector** 25–38
에셋 참조 분석 플러그인 개발과 검증

03 **Graphics Study Archive** 39–62
그래픽스 학습·구현과 AI 아카이브 자동화

전체 목차 →

영상·문서·검증 자료 ↗

# 전체 포트폴리오 목차

프로젝트와 주요 사례를 선택하면 해당 본문으로 이동합니다.

01

## Project Stellar

전투·AI 구현과 문제 해결

**03–24쪽 · 프로젝트 시작 →**

- 프로젝트 구성·개요·플레이 범위 03–05
- 적 인식 지연 · 검증 06–08
- 후방 피격 후 타깃 유지 · 검증 09–11
- 가드 종료 중 피격 · 검증 12–14
- 전투·AI·디버그 구현 구조 15–24

**저장소 ↗** **영상·근거 ↗**

02

## Asset Reference Inspector

에디터 도구 개발과 검증

**25–38쪽 · 프로젝트 시작 →**

- 프로젝트 구성·개요·사용 흐름 25–27
- 필터 입력 오류 · 검증 28–30
- 미사용 후보 탐색 · 검증 31–33
- 구현 구조·CSV·AI 리뷰 34–38

**저장소 ↗** **영상·근거 ↗**

03

## Graphics Study Archive

그래픽스 학습·구현과 자동화

**39–62쪽 · 프로젝트 시작 →**

- 개요·색인·자동화 과정 39–42
- 오류 창 자동화 · 검증 43–45
- FBX 구조 호환 · 검증 46–48
- 그래픽스 학습·구현 범위 49–62

**저장소 ↗** **영상·근거 ↗**

본문의 영상·근거 링크에서 관련 자료를 확인하고, PDF 책갈피로 프로젝트 사이를 이동할 수 있습니다.

# Project Stellar · 상세 목차

항목을 선택하면 해당 설명 페이지로 이동합니다. 관련 영상은 각 페이지에서 확인할 수 있습니다.

## 프로젝트 소개 04–05

**프로젝트 개요 →** 04

개발 환경과 담당 구현 범위를 소개합니다.

**플레이 가능 범위 →** 05

공격·방어와 다수 적 전투를 시연합니다.

## 문제 해결·검증 06–14

**다수 AI 인식 지연 →** 06–08

인식 지연의 원인과 개선 결과를 설명합니다.

**후방 피격 후 타깃 유지 →** 09–11

타깃 만료의 원인과 TTL 수정을 설명합니다.

**가드 종료 중 피격 전환 →** 12–14

가드 정리 오류의 원인과 수정을 설명합니다.

## 구현 구조 15–24

**공격·피격 처리 →** 15–16

공격 충돌부터 피격 반응까지 설명합니다.

**실행 중단·반응 전환 →** 17–18

기존 실행 정리와 새 반응 시작을 설명합니다.

**패링 판정·결과 전달 →** 19–20

패링 판정과 양측의 후속 처리를 설명합니다.

**전투 참여 역할 배정 →** 21–22

여러 적의 역할 배정과 행동 결정을 설명합니다.

**디버그 오버레이 →** 23–24

전투 상태를 관찰하는 도구 구조를 소개합니다.

본문 우측 상단에서 관련 영상을 열거나 색인으로 돌아올 수 있습니다.

# 프로젝트 개요

가드·패링·회피와 다수 적 전투를 구현한 UE5.4 기반 근접 액션 프로젝트입니다.

개발 기간

**2025.12.01 ~ 현재**

2026.10.21 기준

개발 규모

**1인 개발 · AI 협업**

개발 환경

**Unreal Engine 5.4**

C++ / Blueprint

프로젝트 범위

**전투·AI·피드백 구현**

에셋 설정·게임플레이 통합

## 핵심 구현 범위

01

### 전투 처리와 실행 관리

피해·방어 판정과 액션·리액션의
실행 결정, 중단 및 전환을 처리합니다.

공격·피격·가드·패링·회피

02

### AI 인지와 전투 참여

인지·피격 근거에 따라 대상을 판단하고
교전·경계·관찰 역할을 배정합니다.

대상 판단 · 행동 선택 · 참여 역할

03

### 전투 피드백과 검증

실행 시점에 시각·음향 피드백을 연결하고
디버그 오버레이로 상태를 확인합니다.

전투 연출 · HUD · 디버그 도구

## 외부 에셋 활용 범위

캐릭터·모션·무기·환경·VFX·효과음 활용
폰트·아이콘의 출처와 제작 방식 구분

외부 에셋 출처 보기 ↗

## 구현 범위·AI 협업

전투·AI·피드백·HUD·디버그 도구 구현
외부 에셋 가공·설정 및 게임플레이 통합
본인: 요구·검토 / Codex: 구현·검증 지원

구현 구조 상세 보기 ↗

# 실제 플레이로 보는 전투

공격과 방어를 연결한 반격, 균형 붕괴 후 처형, 다수 적과의 교전을 구현했습니다.




## 다수 적 전투 시연 ↗

여러 적의 교전 참여와 실제 공격을 확인할 수 있습니다.

## 공격과 방어

콤보 공격과 가드·패링·회피로
상대의 공격에 대응합니다.

## 반격과 처형

회피 후 카운터 러시로 반격하고,
균형이 무너진 적을 처형합니다.

## 다수 적과의 교전

여러 적이 접근하는 상황에서
교전 참여와 주변 경계가 이어집니다.

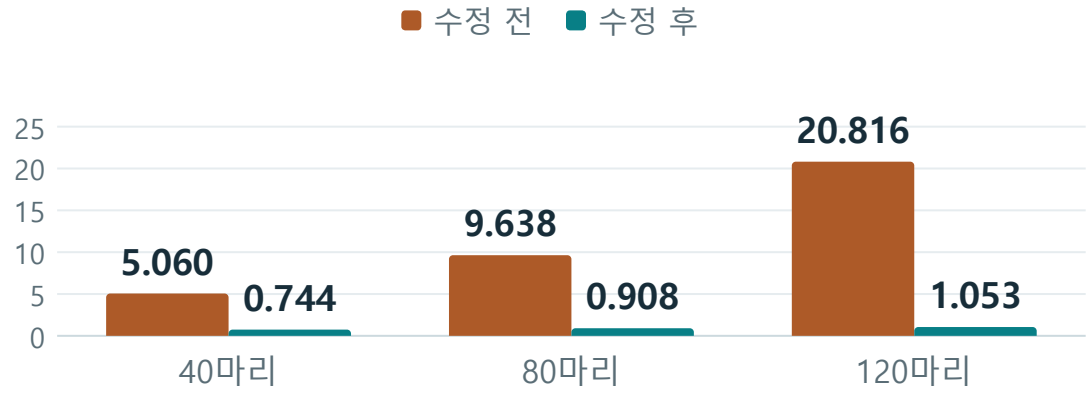
관련 기능 시연

회피·반격 ↗ 균형·처형 ↗ 무기·피드백 ↗

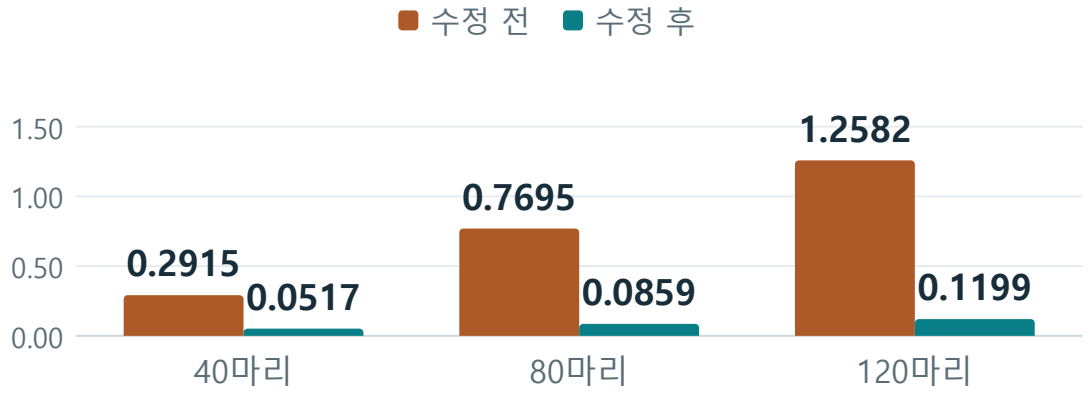
# 문제: 다수의 적이 배치된 환경에서 대상 인식 지연

플레이어를 감지해 대응하도록 구성했지만, 적 수가 늘어날수록 최초 인식이 늦어졌습니다.
40·80·120마리에서 인식 로그와 프로파일링 CSV를 수집해, 인식 지연과 감지 처리 비용을 비교했습니다.

## 최초 유효 대상 인식 지연 · p95 (초)




## AIPerception 처리 시간 · p95 (ms)




**최초 유효 대상 인식 지연** | AI별 계측 시작부터 플레이어의 첫 성공 감지 콜백까지 걸린 시간

**AI Perception 처리 시간** | 게임 스레드의 AI Perception 처리에 사용한 프레임별 시간

**측정 조건** | 40·80·120마리 · 전후 각 3회 · 회당 40초 · 동일 배치·카메라 · 녹화 없이 측정

**측정 환경** | UE 5.4.4 · Win 11 · Ryzen 9 7950X3D · RTX 4070 Ti · D3D12 · Development

**집계 방식** | 실행별 p95의 3회 중앙값
반복 범위·정상 인식은 8쪽

## 40~120마리에서 최초 인식 지연을 약 0.74~1.05초로 단축

인식 지연·감지 처리 비용 감소. 전체 프레임 시간 개선은 확인되지 않았습니다.

측정: 2026.10.02 · p95: 95백분위수 · 영상 13은 측정과 별도의 동작 비교 시연

# 해결: 같은 편 Enemy를 감지 후보에서 제외

같은 편 Enemy도 Sight 감지를 거친 뒤 제외되어, 불필요한 감지 처리가 발생했습니다.
Player는 적대, Enemy는 아군으로 분류해 같은 편을 감지 단계에서 제외했습니다.

## 원인 관찰: 감지 후 제외되는 같은 편 Enemy

120마리 · AI별 관측값의 p95 · 3회 중앙값

| | 누적 감지 객체 수 | 같은 편 Enemy | 최대 등록 타깃 수 |
|---|---|---|---|
| 수정 전 | 111 | 110 | 1 |
| 수정 후 | 1 | 0 | 1 |

## 원인: 같은 편까지 감지한 뒤 제외

| | Sight 감지 | 감지 콜백 | 전투 타깃 등록 |
|---|---|---|---|
| 수정 전 | Player + Enemy | **Enemy 제외 (Provider 검사)** | Player만 등록 |
| 수정 후 | 팀 관계로 Enemy 감지 제외 | **Player만 전달** | Player만 등록 |

## 수정: 팀 관계를 명시해 Sight 필터 적용

ACAIController::
**GetTeamAttitudeTowards(...) const**

```
if (Other.IsA<ACPlayer>())
    return ETeamAttitude::Hostile;
if (Other.IsA<ACEnemy>())
    return ETeamAttitude::Friendly;
```

Sight: 적대 감지 ON · 아군/중립 OFF

## 수정 핵심: 감지 단계에서 같은 편 제외

같은 편 Enemy가 감지 콜백으로 넘어오기 전에 제외합니다.
Player의 전투 타깃 등록 흐름은 유지합니다.

## 변경 효과

전투 대상이 될 수 없는 후보의
불필요한 감지·콜백 처리 감소

# 검증: 반복 측정에서도 지연 감소·전원 인식 완료

수정 전후 각각 3회씩 측정해 모든 AI의 플레이어 인식 완료를 확인했습니다.
수정 후 인식 지연의 반복 측정 범위와 전체 프레임 시간도 함께 확인했습니다.

## 수정 후 인식 p95 · 반복 측정 범위

| 개체 수 | 중앙값 | 3회 최소–최대 | 전후 매회 완료 |
|---|---|---|---|
| 40마리 | 0.744초 | **0.744–0.745초** | 40 / 40 |
| 80마리 | 0.908초 | **0.904–0.930초** | 80 / 80 |
| 120마리 | 1.053초 | **1.049–1.067초** | 120 / 120 |

측정 기준: 40·80·120마리 / 전후 각 3회 / 각 40초 / 녹화 없이 측정

## 정상 인식과 감지 처리 비용

**등록 타깃 수**

| 수정 전 | | 수정 후 |
|---|---|---|
| 1 | → | 1 |

**AI Perception 처리 시간**

| 수정 전 | | 수정 후 |
|---|---|---|
| 1.2582 ms | → | 0.1199 ms |

120마리 · 실행별 p95의 3회 중앙값

### 검증 결론

3회 모두 플레이어 인식 완료
수정 후 지연의 반복 측정 범위 확인

### 비교·해석 조건

p95는 각 실행에서 집계
개별 값은 검증 상세 참고

### 검증 범위·제한

전체 프레임 p95 개선 없음
20.7365 → 21.1009 ms

# 문제: 후방 피격 후 공격자를 추적하지 못함

후방에서 피격된 적이 리액션을 마친 뒤에도 공격자를 향해 돌아보지 않았습니다.
공격자 타깃 유지 여부와 종료 후 추적 회전을 같은 환경에서 비교했습니다.

## 수정 전 · 기존 정책 재현



리액션 종료 후 · 회전 없음

**피격 근거 만료**

**전투 참여 링 없음**

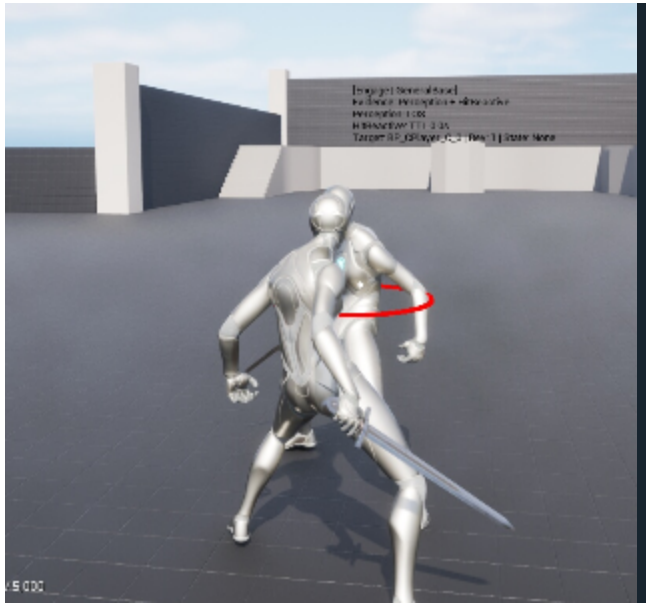
동일 실행 · 종료 직후 로그

Reaction=None

**Target=None**

**Policy=NoCombatTarget**

## 수정 후 · 변경 정책



리액션 종료 후 · 추적 회전

**공격자를 향해 회전**

**전투 참여 링 유지**

동일 실행 · 종료 직후 로그

Reaction=None

**Target=BP_CPlayer_C_0**

**Policy=Tracking**

## 피격 리액션 종료 시 타깃 없음

TTL이 먼저 만료되어 타깃 해제
후속 Investigate는 비교에서 제외

## 피격 리액션 종료 시 타깃 유지

종료 직후 타깃과 포커스를 적용하고
공격자를 향해 회전하는 것을 확인

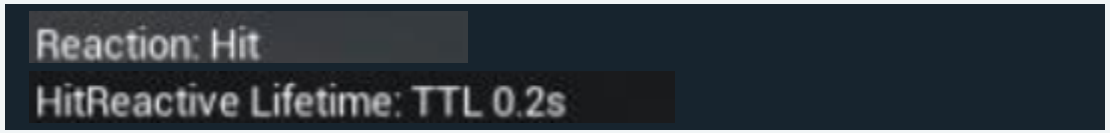
동일 에셋·TTL(피격 근거 유지 시간) 1초 · 10월 1일 보존 환경 재현 · 후속 Investigate 제외

# 해결: 피격 리액션 종료 후 TTL 감소 시작

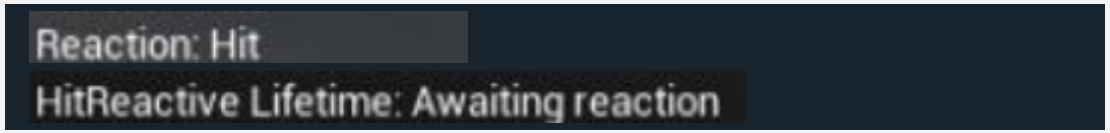
피격 근거의 유지 시간(TTL)이 리액션 중 소모되어, 종료 시 공격자 타깃이 사라졌습니다.
근거 등록은 피격 수락 시 유지하고, TTL 감소 시작을 해당 리액션 종료 이후로 옮겼습니다.

**관찰: 피격 진행 중 TTL 상태** 재촬영 진단 오버레이 발췌

**수정 전 · 피격 중 TTL 감소**




**수정 후 · 피격 종료까지 TTL 시작 대기**




## 원인: 리액션 종료 전에 피격 근거의 TTL 소진

| | 피격 시작 | 피격 진행 중 | 리액션 종료 추적 연결 |
|---|---|---|---|
| 수정 전 | TTL 시작 → | TTL 만료 타깃 해제 → | **TTL 만료 타깃 없음** |
| 수정 후 | 타깃 등록 TTL 시작 대기 → | 타깃 유지 TTL 시작 대기 → | **TTL 시작 추적 연결** |

## 수정: 피격이 끝난 뒤 TTL 시작

① 적의 **피격 리액션 종료** 확인

② 해당 피격 당시 **저장한 공격자** 확인
ResultSerial로 피격과 공격자를 연결

③ 해당 공격자에 대한 **HitReactive TTL 시작**

```
CombatParticipationComp_Injected->
    StartHitReactiveEvidencePostReactionTTL(
        combatantTarget->Get(), resultSerial);
```

## 수정 핵심: 근거 등록과 TTL 시작 분리

피격 근거는 수락 시 등록하고,
TTL은 해당 리액션 종료 후 시작합니다.

## 연결 기준

ResultSerial로 피격·공격자 연결
연속 피격·종료 순서는 11쪽에서 검증

# 검증: 늦거나 중복된 종료 알림에도 기록이 제때 만료되는지 확인

적은 자신을 때린 공격자를 기억해 타깃 판단에 사용합니다. 이 기록은 마지막 피격 반응 종료 후 1초간 유지됩니다.
연속으로 맞았을 때 이전 종료 알림이 늦거나 중복해서 도착해도, 기록의 유지 시간이 늘어나지 않는지 확인했습니다.

## ① 같은 공격자에게 두 번 연속으로 맞은 경우

아래 세 행은 0.50초 → 0.70초 → 1.60초 순서로 이어지는 하나의 검사입니다.

| 시점 | 어떤 상황을 만들었나? | 무엇을 확인했나? |
|---|---|---|
| 0.50초 | 두 번째 피격 반응을 종료했습니다. | **1초 뒤인 1.50초에 기록이 만료되도록 설정됐습니다.** |
| 0.70초 | 첫 번째 피격의 종료 알림을 뒤늦게 전달하고, 두 번째 피격의 종료 알림도 다시 전달했습니다. | **만료 시각은 1.50초 그대로였습니다. 1.70초로 늘어나지 않았습니다.** |
| 1.60초 | 만료 시각이 지난 뒤 기록을 확인했습니다. | **해당 공격자에게 맞았다는 기록이 만료됐습니다.** |

## ② 서로 다른 공격자에게 맞은 경우 · 별도 검사

A의 기록이 만료돼도, 아직 시간이 남은 B의 기록은 유지돼야 합니다.

| | | |
|---|---|---|
| 1.25초 | A 기록은 1.20초, B 기록은 1.50초에 만료되도록 설정한 뒤 확인했습니다. | **A의 기록만 만료되고, B의 기록은 유지됐습니다.** |

## 자동 검사와 실제 시연

### 자동 검사 · NullRHI

**4그룹 · 19시나리오 모두 통과**

이전 종료·요청 거절
중단·공격자별 독립 만료

### 실제 시연

**피격 반응 후 대상 추적 복귀**

Tracking 상태 전환 확인

기록 유지와 현재 타깃 선택은 별개

## 검증 결론

늦거나 중복된 종료에도 만료 시각 유지
공격자별 기록은 각자의 시각에 만료

## 비교·해석 조건

수정 전은 정책 복원 시연
당시 바이너리를 사용하지 않음

## 검증 범위·제한

공격자 소멸 후 대기 기록
정리 보완안 원본 미반영

# 문제: 가드 종료 중 피격 모션·이펙트 누락

가드 해제 중에도 피격 반응이 이어져야 했지만, 피해만 적용되고 모션·이펙트가 누락됐습니다.
실행 화면과 로그를 대조해, 피해 적용 후 피격 반응이 시작되지 않는 구간을 확인했습니다.

## 수정 전




**화면 관찰**

① 피격 모션 전환 없음
피격 반응 효과 없음

실행 로그 · 핵심 필드 발췌

Damage=20.000
Stage=InterruptGuardOut

[ReactionDecision]
**Overlay handling failed.**

## 수정 후




**화면 관찰**

① 피격 모션 전환
② 피격 반응 효과 표시

실행 로그 · 핵심 필드 발췌

Damage=20.000
Stage=InterruptGuardOut

보조 근거 · 실행 로그
**동일 실패 로그 없음**

## 피해 적용 후 피격 모션·이펙트 누락

가드 중단까지 진행했으나
피격 리액션 시작 전 실패

## 피해 적용 후 피격 모션·이펙트 재생

가드 중단 후 피격 리액션으로 전환
피격 모션과 이펙트 재생 확인

① 피격 자세 / ② 피격 반응 효과 · 검 궤적과 구분 · 연속 영상에서 확인: 영상 11 · 00:05–00:23

# 해결: 가드 상태 정리가 끝났어도 피격 전환 허용

가드 중단과 피격 시작 준비에서 가드 상태 정리가 연속으로 요청됐습니다.
이미 정리된 경우에도 요청을 수락하도록 수정해 피격 전환이 중단되지 않게 했습니다.

## 호출 흐름과 실패 지점

**① 기존 가드 중단 처리**
ClearGuardState 완료

↓

**② 새 피격 리액션 시작 준비**
ClearGuardState 재요청

**수정 전 · 요청 거절 (false)**
가드 상태가 이미 없음
Hit 시작 전 처리 종료

**수정 후 · 요청 수락 (true)**
이미 정리된 상태도 허용
Hit 시작 단계로 진행

실제 실패 로그 · 타임스탬프 생략

```
[OverlayHandling] Failed Handling=
EObservableOverlayHandling::ClearGuardState
```

## 수정한 수락 조건

UCDefenseComponent::
**CanApplyOverlayHandling(...) const**

```
// ...
case EObservableOverlayHandling::
    ClearGuardState:
- return HasGuardRuntimeState();
+ return true;
// ...  관련 분기 발췌   - 수정 전 / + 수정 후
```

## 변경 범위: 정리 요청의 수락 조건

가드 상태 유무로 요청을 거절하던 분기를 수정했습니다.
실제 상태 제거 호출은 유지합니다.

## 수정 핵심: 이미 정리된 상태도 정상 처리

같은 정리 요청이 다시 들어와도
새 피격 반응의 시작을 막지 않도록 합니다.

## 적용 범위

ClearGuardState 정리 요청
다른 실행 허용 조건은 별도 판단

# 검증: 가드 종료 중 피격 후 다음 가드까지 실행

가드를 내리는 도중 공격을 맞았을 때, 가드 정리 과정에서 막히지 않고 피격 반응이 실행되는지 확인했습니다.
피격 반응 종료 후 기본 이동 속도 설정이 복원되고, 다시 가드를 시작·유지·종료할 수 있는지 확인했습니다.

## 실제 실행: 하나의 연속된 검사

모든 시간은 처음 가드 해제를 요청한 순간부터 계산했습니다.

| 경과 시간 | 입력·발생한 일 | 확인한 결과 |
|---|---|---|
| 0.000초 | 가드 해제를 요청했습니다. | **가드를 내리는 종료 동작이 시작됐습니다.** |
| 0.076초 | 가드를 내리는 도중<br>적의 공격에 맞았습니다. | **남은 가드 정리 요청 수락 후 피격 반응 실행**<br>**HP 5,000 → 4,700** |
| 1.292초 | 피격 반응 종료와 속도 설정 확인 후,<br>다시 가드를 요청했습니다. | **기본 이동 속도 설정 600 확인**<br>**새 가드 시작 동작 실행** |
| 2.806초 | 새 가드의 유지 상태를 확인한 뒤,<br>종료 동작으로 전환됐습니다. | **가드 유지 상태 진입 확인**<br>**이어서 가드 종료 동작 시작** |
| 4.814초 | 두 번째 가드가 끝난 뒤<br>상태를 확인했습니다. | **기본 이동 속도 설정이**<br>**다시 600인지 확인했습니다.** |

### 실제 실행과 별도 자동 검사

**실제 실행 · 격리 PIE**

**피격 반응 → 다음 가드**
**시작·유지·종료 확인**

Run 600 속도 설정 복원 확인
달리는 동작의 재개 검사는 아님

**별도 자동 검사**

**24조건 × 25회**
**정리 호출 총 600회**

각 조건 종료 시 가드 상태 해제
기본 이동 속도 설정 복원 확인

요청은 재현 코드에서 실행 · 격리 PIE | 이동 속도: PIE Run 600 / 자동 검사 Run 420·Sprint 620 | 자동 조건: 가드 상태 4 × 정리 경로 3 × 이동 설정 2

## 검증 결론

피격 반응·기본 이동 속도 설정 복원 확인
다음 가드와 반복 정리 후 상태 복원 확인

## 비교·해석 조건

기존 시연 피해 20 · 본 검증 300
자동 검사와 PIE 이동 설정 구분

## 검증 범위·제한

Sprint 유지·800 복원 미확인
수동 PIE 이펙트 동등성 미확인

# 공격 실행과 피격 반응의 연결

Player의 일반 공격이 Enemy에 적중한 장면입니다.
공격 실행, 충돌과 결과 판정, 피격 반응의 세 단계로 구성됩니다.




### 1 공격 실행

Player의 입력에 따른
공격 동작 실행

### 2 충돌과 결과 판정

무기 충돌 데이터 전달 후
Enemy의 피해·방어 결과 판정

### 3 피격 반응

판정 결과에 따른
Enemy의 피격 반응

**공격자는 공격 실행과 충돌 데이터 전달을, 피격자는 결과 판정과 반응을 담당**

영상 01 · 00:09–00:31 | 공격 실행 → 피해 적용 → 피격 반응

# 공격·피격 처리의 책임과 실행 흐름

일반 타격은 공격 실행(Action), 충돌·결과 판정, 피격 반응(Reaction)의 순서로 처리됩니다.
공격자는 충돌 데이터를 전달하고, 피격자는 자신의 상태로 피해·방어 결과와 반응을 결정합니다.



일반 타격 · Player → Enemy

**설계 이유**

**피해 적용과 피격 반응은 별개의 판단**

피해 적용 여부와 피해량은 피격자의 방어 상태와 공격 속성으로 결정합니다. 피격 반응은 현재 실행 중인 액션·리액션과 전환 조건에 따라 실행 여부를 결정합니다.

## ① 공격 실행 · 왼쪽부터 →

공격자 Player
**행동 판단**
현재 상태 기반 정책 평가 요청
ActionOrchestrator

➡

공격자 Player
**실행 관리**
현재 실행 관리·판단 결과 적용
ActionComponent

➡

공격자 Player
**공격 실행**
Montage 재생·종료 정리
Action

Notify로 무기 충돌 활성화 ⬇

## ② 충돌·결과 판정 · ← 오른쪽부터

공격자 Player
**무기 충돌**
공격 판정 구간 내 충돌 발생

⬅

공격자 Player
**충돌 검증·전달**
공격 판정 구간·중복 충돌 확인
CombatSignalSource

⬅

피격자 Enemy
**피해·방어 판정**
피격자 상태로 결과 결정
CombatSignalTarget

⬇ 판정 결과로 반응 요청

## ③ 피격 반응 · 왼쪽부터 →

피격자 Enemy
**반응 판단**
현재 상태 기반 정책 평가 요청
ReactionOrchestrator

➡

피격자 Enemy
**실행 관리**
현재 반응 관리·판단 결과 적용
ReactionComponent

➡

피격자 Enemy
**피격 반응**
Montage 재생·종료 정리
Reaction

# 공격 중 피격에 따른 실행 전환

Enemy가 공격 도중 Player의 타격을 받아 피격 자세로 전환하는 장면입니다.
공격 진행, 피격에 따른 공격 중단, 피격 반응 시작의 세 단계로 구성됩니다.

## 1 공격 진행



Enemy가 진행 중인 공격 동작

Enemy HP 200 / 200

## 2 피격·공격 중단



Player의 타격으로 Enemy 공격 중단

Enemy HP 195 / 200

## 3 피격 반응 시작



Enemy가 공격에서 피격 자세로 전환

Enemy HP 195 / 200

**중단이 허용된 공격은 기존 실행을 정리한 뒤 피격 반응으로 전환**

영상 02 · 00:05–00:28 | 은색 Enemy의 공격·피격 상태와 종료 후 복귀 확인

# 실행 중단 조건과 피격 반응의 시작 순서

공격 중 피격된 Enemy가 실행 중인 공격을 중단하고 피격 반응으로 전환하는 구조입니다.
반응의 중단 요구와 공격의 중단 허용을 함께 확인하고, 기존 공격을 정리한 뒤 새 반응을 시작합니다.



사례 · 공격 중 피격된 Enemy

**설계 이유**

**이전 공격 상태가 새 반응에 남지 않게 정리**

피격 반응 중에 이전 공격의 충돌 판정이 남지 않도록, 충돌·트레일과 실행 정보를 정리한 뒤 반응을 시작합니다. 개별 피드백은 각자의 종료 기준에 따라 따로 종료합니다.

## ① 중단 조건 확인·요청 · 왼쪽부터 →

**피격 반응의 중단 요구**
현재 공격의 중단을 요구하는 반응인지 확인
Reaction.WantIntervention

→

**현재 공격의 중단 허용**
해당 피격 반응에 의한 중단 허용 여부·시점 확인
Action.AllowIntervention

→

**공격 중단 요청**
두 조건이 충족되면 현재 실행 중인 공격에 요청
ReactionOrchestrator / Component

↓

## ② 실행 중인 공격 정리 · ← 오른쪽부터

**Montage 재생 중지**
실행 중인 공격의 Montage 재생 중지
StopMontage

←

**무기 충돌·트레일 정리**
무기 충돌·트레일 비활성화
무기 전환·충돌 문맥 정리
CleanupRuntimeEffects

←

**공격 실행 정보 초기화**
공격 데이터·허용 구간 초기화
컴포넌트에 종료 알림
ClearRuntime / ActionComponent

↓

## ③ 새 피격 반응 시작

**기존 공격의 정리가 끝나면 Hit Reaction과 Montage 실행**

ReactionComponent / Reaction.Start

# 패링의 피해 차단·반응·균형 변화

Player가 Enemy의 공격을 패링한 장면입니다.
충돌 데이터 전달, 방어 결과 판정, 양측 후속 처리의 세 단계로 구성됩니다.




## Player HP 유지

패링으로 공격을 막아 HP 유지

## Player 패링 반응

판정 결과에 따른 패링 반응

## Enemy의 남은 균형 감소

패링이 누적될 때마다
남은 균형이 1씩 감소
0이 되면 균형 붕괴 반응 요청

**패링은 HP 피해를 차단하고, Player의 반응과 Enemy의 균형 변화로 이어짐**

영상 03 · 00:09–00:34 | 사진: 균형 2→1 / 영상: 3→2 · 처리 규칙은 코드에 근거.

# 패링 판정과 양측 후속 처리

Player가 Enemy의 공격을 패링했을 때의 처리 구조입니다.
충돌 데이터를 받아 방어 결과를 판정하고, Player의 피해·반응과 Enemy의 남은 균형에 반영합니다.

## ① 충돌 발생

공격자 Enemy

**공격 충돌 발생**

공격 구간·중복 충돌 확인
Player에 충돌 데이터 전달

CombatSignalSource

설계 이유

**피해가 없어도 후속 처리는 필요**

방어자 반응을 요청하고 공격자 균형을 줄입니다. 반응 실행 여부는 현재 실행 상태·전환 조건으로 결정합니다.

## ② 피격 조건과 방어 결과 판정

피격자 Player

**충돌 데이터 수신**
TakeDamage로 충돌 데이터 수신

**피격 조건 확인**
생존·피격 허용 상태·공격 속성 확인

**방어 결과 결정**
회피·패링·가드 조건 평가

**이 사례의 판정: Parry**

타격 수락 · 최종 피해량 0

CombatSignalTarget / Defense

## ③ 판정 결과의 양측 반영

Player · 방어자 처리

**피해 처리**
HP 차감 없음
피해량 0

**패링 반응 요청**
정책 허용 시 실행
ReactionOrchestrator

자신의 피해 확정 후 패링 반응 요청

Enemy · 전달받은 결과 처리

**남은 균형 감소**

패링 누적 횟수 +1 · 남은 균형 -1

CombatResultPacket / Balance

# 같은 대상을 둔 다수 적의 전투 역할

여러 Enemy가 같은 Player를 대상으로 서로 다른 참여 역할을 배정받은 장면입니다.
참여 근거 수집, 참여 역할 배정, 개별 행동 결정의 세 단계로 구성됩니다.



### Engage · 교전 3명

기본 2명 + 피격 근거 추가 1명

- 피격 근거 없는 Engage
- 인식·피격 근거가 있는 Engage
- 피격 근거만 있는 Engage

### Alert · 경계 1명

거리 조건에 따라 접근·경계

### Observe · 관찰 1명

대상을 관찰하는 역할

### None · 미배정

현재 참여 역할 없음

## 인식·피격 근거와 참여 상한에 따라, 같은 대상을 둔 적들의 역할을 배정

영상 06 · 00:33–01:25 | 사진: 기본 2+추가 1·근거별 링 / 영상: 기본 1+추가 1·역할 표시

# 참여 근거 수집과 역할 배정·행동 결정

참여 관리 Subsystem은 인식·피격 근거를 모아 대상별 참여 상한 안에서 역할을 배정합니다.
개별 AI는 현재 실행 상태를 먼저 확인하고, 배정 역할과 거리에 따라 행동을 결정합니다.



## 장면의 역할 배정

| | 배정 / 상한 |
|---|---|
| 교전 Engage | 3 / 3명 |
| 경계 Alert | 1 / 1명 |
| 관찰 Observe | 1 / 1명 |

사진: 기본 2+추가 1 / 영상 06: 1+1

설계 이유

**참여와 행동 판단 분리**

참여 인원을 대상별로 제한하되, 역할만으로 행동을 정하지 않고 각 AI의 거리·실행 상태에 맞춰 추적·공격 등을 결정합니다.

### ① 근거 수집·후보 구성 · 왼쪽부터 →

**참여 근거 수집**
시야·기억: Perception
직접 피격: HitReactive
가드·패링 결과도 피격 근거에 포함

→

**참여 근거 병합**
같은 Enemy·대상 조합의 인식·피격 근거를 통합

→

**참여 후보 구성**
유효 근거·대상 유효성 확인
대상 우선순위·거리로 정렬

### ② 대상별 역할 배정 · ← 오른쪽부터

**기존 역할 유지**
유효 근거·배정 유지 조건 확인

←

**승격·신규 배정**
역할별 빈자리와 근거로 배정

←

**배정 결과 전달**
대상·역할·기본/추가 참여 전달

배정 개념: 기본 참여 우선. 기본 상한 이후에는 피격 근거가 있는 후보만 추가 배정.

### ③ 역할별 행동 선택

현재 실행 상태를 먼저 확인한 뒤, 배정 역할과 거리로 행동 선택

**Engage · 교전 / 추격**
멀면 추적, 가까워지면 공격 조건에 따라 공격

**Alert · 경계 / 추격**
멀면 추적, 가까워지면 경계 행동 수행

**Observe · 대상 관찰**
공격에 참여하지 않고 대상을 관찰

# 화면 패널과 월드 도형으로 확인하는 전투 상태

캐릭터의 상태·처리 기록과 전투 공간의 역할·방향·범위를 함께 확인하는 디버그 도구입니다.
캐릭터 상세·이벤트 로그·월드 요약의 화면 패널과 월드 디버그 드로우로 구성됩니다.




### ① 캐릭터 상세

Player·관찰 Enemy의
현재 상태와 최근 판단 정보

### ② 이벤트 로그

실행 판단·전투 판정·AI 처리 등
발생한 이벤트의 이력

### ③ 월드 요약

최근 주요 처리 결과와
대상별 전투 참여 현황

### ④ 월드 디버그 드로우

전투 참여 역할·공격 방향·앵커 등
공간 정보를 도형으로 확인

## 상태·처리 이력은 패널로, 역할·방향·범위는 월드 도형으로 확인

영상 10 · 00:13–01:07 | 사진은 참여 링·텍스트. 방향·앵커 도형은 해당 기록·표시 조건에서 출력.

# 관찰 데이터를 화면 패널과 월드 도형으로 출력

컴포넌트·Subsystem의 현재 상태와 이벤트·진단 기록을 관찰 데이터로 사용합니다.
화면 패널은 텍스트를 구성해 출력하고, 월드 도구는 위치·방향·범위를 도형으로 그립니다.



화면 패널과 월드 디버그 드로우

**설계 이유**

**현재 상태와 사건 기록을 함께 확인**

현재 상태만으로는 과거 판단의 이유를 알기 어려워 당시의 정보를 따로 기록합니다.
피격 반응과 대상·TTL을 대조해 반응 중 대상이 만료된 상황을 확인했습니다. (9–11쪽 사례)

**HUD · 디버그 출력의 진입점**

관찰 대상을 확인하고 상태를 조회한 뒤, 화면 패널과 월드 드로우를 각각 호출

**관찰 데이터**

**현재 상태 조회**

실행·역할·위치 정보

Component / Subsystem

**이벤트·진단 기록**

판단 시점의 정보 보관
현재 상태와 구분해 조회

SnapshotStore

**출력 A · 화면 패널**

**정보·텍스트 구성**

현재 상태·이벤트·요약 구성

ViewDataBuilder / TextFormatter

→

**패널 그리기**

화면 배치·텍스트 출력

CanvasRenderer

**출력 B · 월드 디버그 드로우**

**기능별 공간 정보 구성**

역할·좌표·방향·반경

기능별 Debug Helper

→

**월드 도형 그리기**

링·화살표·범위·텍스트

DrawDebug Helpers

월드 표시 예: 참여 역할 링 · 피격 근거 앵커의 기준점·범위 · Attack Facing의 방향·보정 범위

# 프로젝트 구성

항목을 선택하면 해당 설명 페이지로 이동합니다. 관련 영상과 근거는 각 페이지에서 확인할 수 있습니다.

## 프로젝트 소개 26–27

**프로젝트 개요 →** 26

개발 목적과 환경, 작업 범위를 소개합니다.

**실제 사용 흐름 →** 27

에셋 선택부터 분석·저장까지 보여줍니다.

## 문제 해결·검증 28–33

**필터 입력 오류 →** 28–30

Enter 뒤 사라지는 필터 값을 수정합니다.

**미사용 후보 탐색 재설계 →** 31–33

선택 관계 밖의 후보도 찾도록 변경합니다.

## 프로젝트 구현·검증 34–38

**AI 협업 사전 준비 →** 34

구현 전에 정한 작업 기준을 소개합니다.

**전체 처리 구조 →** 35

입력부터 분석·출력까지의 구조입니다.

**공유·순환 관계 처리 →** 36

공유는 유지하고 순환 탐색은 종료합니다.

**CSV 저장 기능 →** 37

분석 결과를 파일로 저장하는 기능입니다.

**AI 리뷰 활용 →** 38

코드 리뷰를 수정·확인에 연결한 사례입니다.

# 에셋 참조 관계를 조회·기록하는 에디터 도구

개발 목적: Unreal Editor 도구 제작 학습과 실제 프로젝트의 에셋 참조 구조 분석
적용 범위: 게임 코드에 의존하지 않는 독립 플러그인으로 개발하고 액션 프로젝트에 적용

| 개발 기간 | 개발 규모 | 개발 형태 | 개발 환경 |
| --- | --- | --- | --- |
| **2026.07.13–07.20** | **1인 개발** | **개인 프로젝트** | **UE5.4 · C++ · Slate** |
| 구현 및 최종 적용 기록 기준 | | Codex 활용 · 계획·구현·리뷰·검증 | Asset Registry 기반 Editor Plugin |

## 이 포트폴리오에서 보여주는 경험

### 01 AI 협업 개발 체계

초기 작업 규칙을 정하고, 기능별 계획과 검증 기록을 연결했습니다.

사전 준비·AI 리뷰 (34·38페이지) →

### 02 문제 해결과 검증

입력 오류를 수정하고, 미사용 후보 탐색 범위를 목적에 맞게 바꿨습니다.

대표 문제 해결·검증 (28–33페이지) →

### 03 플러그인 구현

참조 조회·에셋 이동·CSV 저장을 에디터 사용 흐름으로 연결했습니다.

처리 구조·CSV 저장 (35–37페이지) →

### 사용자 작업

개발 목표·작업 단위·문서 운영 기준 제시
기능 판단, 오류 확인, 검증 자료 제공

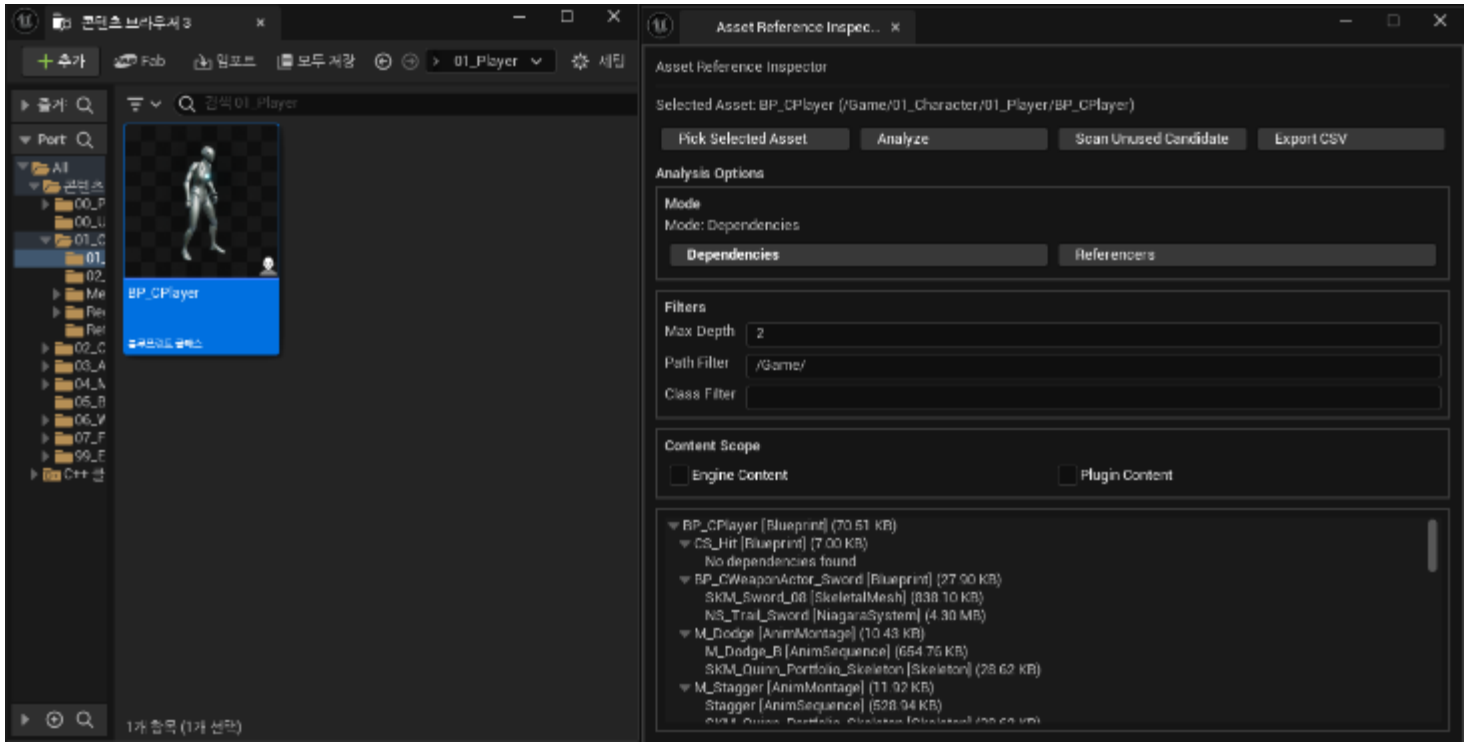
### Codex 작업

계획·문서 구체화와 코드 구현·수정
빌드 실행, 자동 리뷰와 기록 정리

각 사례의 작업은 개발 당시 기록을 기준으로 작성했습니다. 2026.10 추가 QA와 미검증 범위는 개발 당시 기록과 구분합니다.

# 에셋 참조 관계 확인과 결과 저장

선택한 에셋이 사용하는 대상과, 그 에셋을 사용하는 곳을 트리로 조회합니다.
결과 노드에서 원본 에셋으로 이동하고, 현재 분석 결과를 CSV로 저장합니다.




Dependencies: 선택 에셋이 사용하는 대상 / Referencers: 선택 에셋을 사용하는 곳

## 01 분석할 에셋 선택

콘텐츠 브라우저에서 에셋 선택
Pick Selected Asset으로 가져오기

## 02 관계 조회와 원본 이동

방향·깊이·필터를 정하고 Analyze
결과 노드 더블클릭으로 에셋 이동

## 03 분석 결과 저장

Export CSV로 현재 관계 트리 저장
에셋 이름·부모·깊이·모드 기록

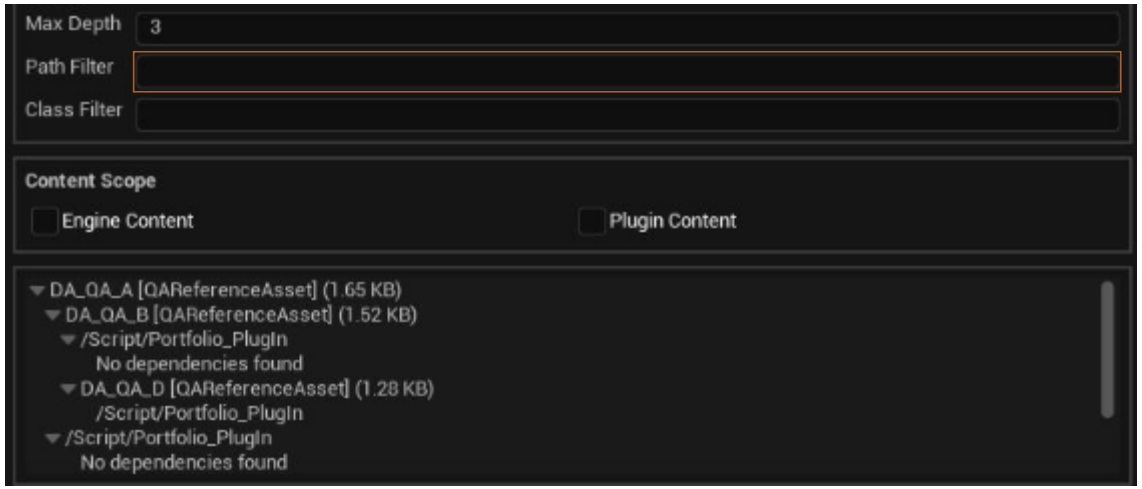
개발 당시 적용 기록 ↗

실제 프로젝트 원본 CSV ↗

# 문제: Enter 확정 뒤 경로가 지워져 필터가 풀림

특정 폴더만 조회하려고 경로를 입력했지만, Enter로 확정하면 필터가 풀렸습니다.

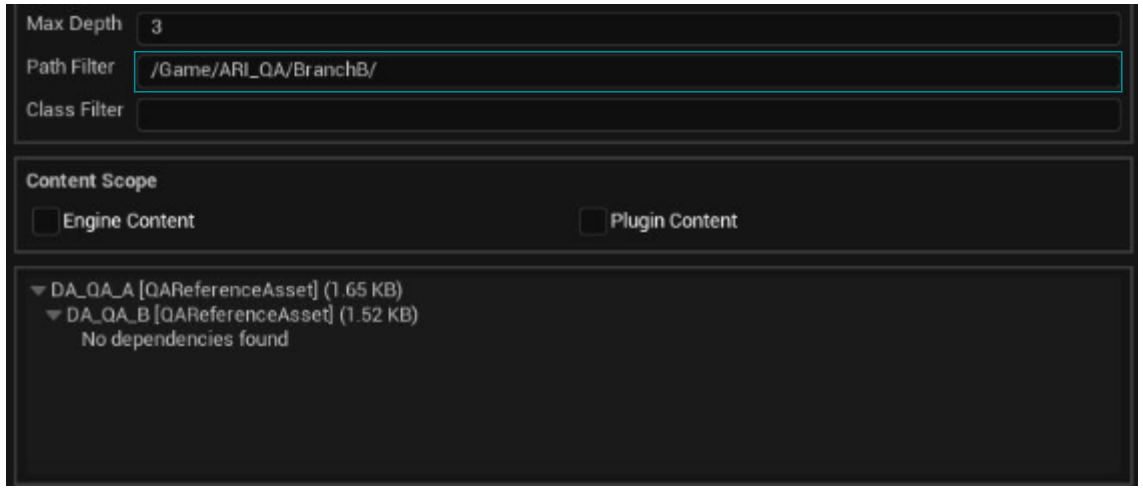
## 수정 전




**① Path Filter가 빈 값으로 바뀜**

**② CSV 10행 · 지정 밖 항목 포함**

## 수정 후




**① 입력한 BranchB 경로를 유지**

**② CSV 2행 · 시작 에셋 A + 필터에 일치한 B**

시작 에셋 A는 필터와 관계없이 포함

추가 QA · 2026.10.21

비교 조건: 이전 입력 처리 코드만 재구성 · 과거 전체 빌드 비교가 아님

이전 처리 CSV ↗

수정 후 CSV ↗

# 해결: Enter를 누르면 사라지던 필터 경로 유지

Enter 확정 직후 호출된 OnCleared 처리에서 저장 경로를 빈 값으로 덮어썼습니다.
OnCleared에서는 저장 경로를 변경하지 않고, Enter·클릭 확정 시 경로를 저장하도록 수정했습니다.

| | ① Enter로 확정<br>호출 사유: OnEnter | ② 후속 호출<br>호출 사유: OnCleared | ③ 분석 결과 |
|---|---|---|---|
| 수정 전 | 입력 경로 저장 | 빈 값으로 덮어씀<br>저장 경로 삭제 | 필터가 풀림<br>지정 밖 항목까지 조회 |
| 수정 후 | 입력 경로 저장 | 저장 건너뛰기<br>입력 경로 유지 | 필터 적용 유지<br>지정 범위로 조회 |

**사용자 제공 로그**

```
CommitType=1, Text='/Game/'
CommitType=3, Text='/Game/'
```

로그·역할 기록 원문 ↗

**수정 전 코드**

```
CommitType == ETextCommit::OnCleared
    ? FString() : InText.ToString();
```

수정 전 코드 원문 ↗

**수정 후 코드**

```
if (CommitType == ETextCommit::OnCleared)
    { return; }
```

수정 후 코드 원문 ↗

**사용자 작업**

오류 발견·초기 설명 재검토 요청
조작별 로그 제공

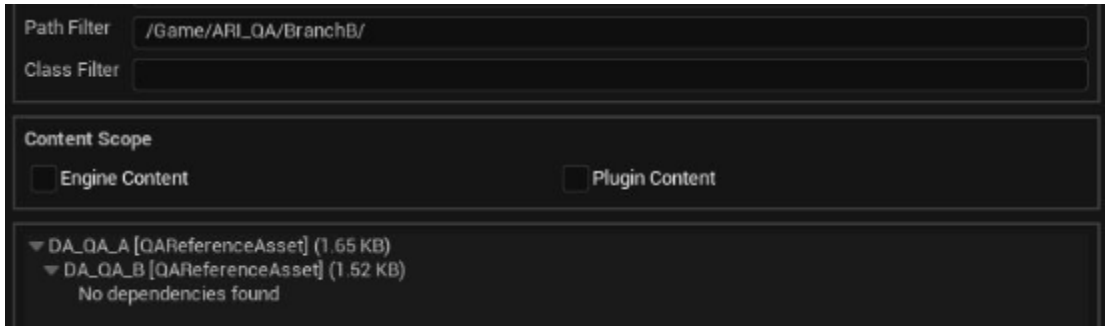
**Codex 작업**

이벤트 로그 추가·원인 분석
OnCleared 처리 수정

# 검증: Enter 확정 경로 유지와 취소·해제 동작 확인

Enter 확정 후 BranchB 경로와 CSV 2행을 확인했습니다. 클릭 확정·Escape 취소·필터 해제도 대조했습니다.

## ① B 폴더 경로 입력 → Enter → 분석




Enter 확정 후 실제 화면 ↗

## ② CSV에도 필터 결과 기록

| AssetName | Path |
|---|---|
| DA_QA_A | /Game/ARI_QA/Root |
| DA_QA_B | /Game/ARI_QA/BranchB |

원본 CSV 발췌 · 2행 모두 표시, 열 2개 선택
A는 필터와 관계없이 포함되는 시작 에셋
Enter 결과 CSV 원본 ↗

## 함께 확인한 기존 입력 동작

| 조작 | 확인된 결과 | 근거 | |
|---|---|---|---|
| C 폴더 경로 입력 → 다른 곳을 클릭<br>입력 확정 | C 경로 확정 · A와 C 출력 | 화면 ↗ | CSV ↗ |
| C 경로 확정 → B 경로로 편집 중<br>Escape | 편집 중인 B 입력 취소 → C 경로 복원<br>CSV도 기존 C 결과와 동일 | 화면 ↗ | CSV ↗ |
| 입력란을 비운 뒤 Enter | 필터 해제 · 분석 결과 10행 | 화면 ↗ | CSV ↗ |

추가 QA · 2026.10.21 · Codex 실행 · 별도 UE5.4 Data Asset 환경
비교 범위: Enter는 이전 입력 처리와 비교 · 클릭·Escape·빈 입력은 수정 후 동작만 확인 →
개발 당시: 사용자 Escape 복원 확인 기록 ↗

미검증: 과거 전체 빌드·다른 UE 버전

추가 QA 화면·영상·이벤트 로그 ↗

# 문제: 미사용 후보를 찾는데 일부 에셋만 검사

목적은 프로젝트에서 사용되지 않는 것으로 보이는 에셋을 찾는 것이었습니다.
하지만 초기 구현은 에셋 하나를 선택한 뒤, 그 에셋의 참조 관계 목록 안에서만 후보를 찾았습니다.

## 기존 | 선택 에셋의 관계만 검사

**프로젝트 콘텐츠의 에셋들**

**초기 검사 범위**

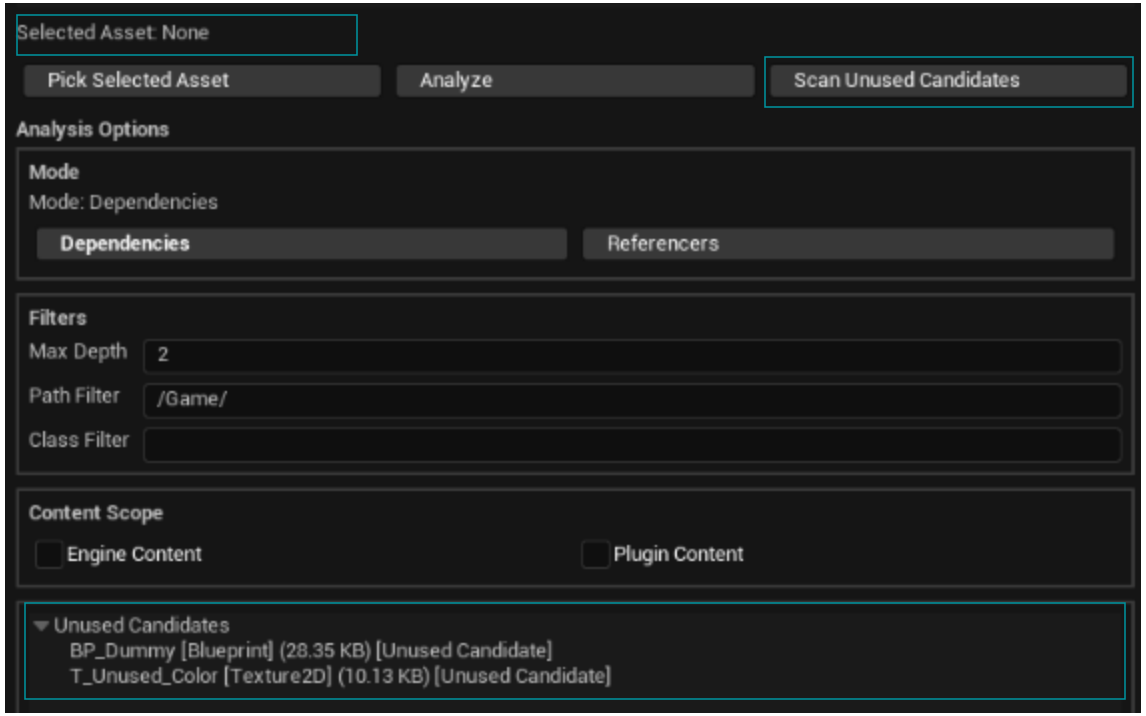
선택한 에셋 ➡ 사용하는 에셋 ➡ 그다음 에셋

**목록에 없는 에셋**

이 범위 밖 에셋은 검사하지 않음

당시 코드로 재구성한 검사 범위 예시

## 변경 | 프로젝트 콘텐츠의 미사용 후보 탐색




초기 구현·변경 기록 ↗

개발 당시 원본 화면 ↗

# 해결: 프로젝트 콘텐츠 전체에서 미사용 후보 선별

선택한 에셋의 관계 목록을 검사하는 대신, 프로젝트 콘텐츠 전체를 모아 후보를 찾도록 바꿨습니다.
기존 관계 조회는 유지하고, 에셋을 선택하지 않아도 실행할 수 있는 탐색 기능으로 분리했습니다.

| | ① 검사 대상 확대 | ② 후보 판정 유지 | ③ 후보 표시 방식 변경 |
|---|---|---|---|
| 초기 | **선택한 에셋의 관계 목록만 검사** | 다른 에셋의 참조가 없는 항목을 후보로 분류 | **참조 관계 트리 안에 미사용 후보 표시** |
| 변경 | **프로젝트 콘텐츠 전체를 검사** | 다른 에셋의 참조가 없는 항목을 후보로 분류 | **미사용 후보만 모아 전용 목록으로 표시** |

구현 범위와 근거

**프로젝트 콘텐츠(/Game) 및 하위 폴더 조회**

검사 범위 구현 ↗
후보 판정 코드·변경 기록 ↗

## 사용자 작업

콘텐츠 전체 검사로 전환 제안·계획 수정 요청
화면 제공·후보 이동 확인

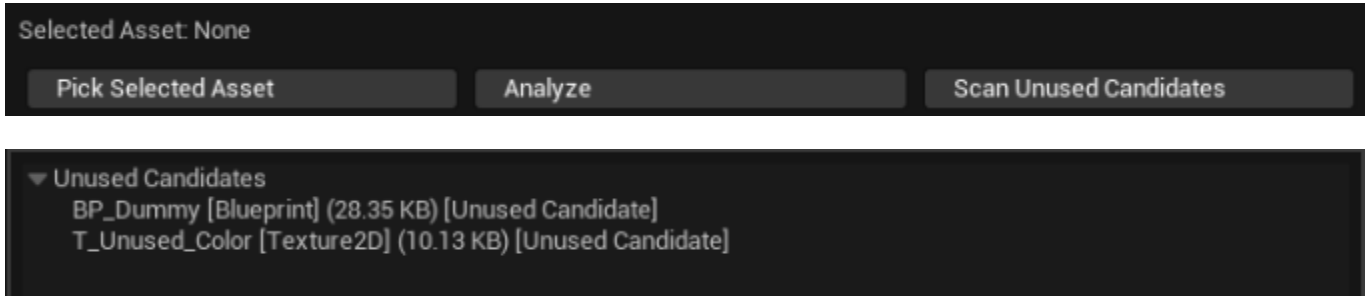
## Codex 작업

계획 재작성·전체 스캔·필터·UI 구현
빌드·검증 기록 정리

# 검증: 에셋 선택 없이 미사용 후보 조회

에셋 선택 없이 후보 2개를 조회하고, 참조받는 에셋의 제외를 확인했습니다.

## 실행 화면 | 선택 상태와 후보 목록




개발 당시 원본 화면 ↗

## 후보 탐색 결과

| | |
|---|---|
| 조회 결과 | **후보 2개 표시**<br>실제 화면 ↗ |
| 제외 확인 | **참조받는 에셋 제외**<br>사용자 확인 기록 ↗ |

## 조회·탐색 편의 기능 확인

| 조건 또는 조작 | 확인 결과 | 근거 |
|---|---|---|
| 특정 폴더만 조회 | T_Unused_Color 1개 | 실제 화면 ↗ |
| 텍스처만 조회 | Texture2D 후보 1개 | 실제 화면 ↗ |
| 일치 항목 없는 경로 조회 | 결과 없음 안내 | 실제 화면 ↗ |
| 후보 더블클릭 | 콘텐츠 브라우저에서 원본 에셋 위치로 이동 | 사용자 확인 기록 ↗ |

개발 당시 검증 · 2026.07 · UE5.4

전체 검증 범위 →

**후보 표시는 삭제 안전성을 보장하지 않습니다.**

미확인: Soft Reference·Asset Manager·동적 로딩 사용 여부
미측정: 대규모 프로젝트 성능

# AI와 함께한 개발 사전 준비

개발 범위와 우선순위를 정하고, 계획 문서와 AI 작업 지침을 마련한 뒤 구현을 시작했습니다.

## ① 개발 범위·우선순위 결정

**범위**

게임 전용 코드와 분리된
독립 에디터 플러그인

**우선순위**

Tree View 우선 · Node Graph 후순위

초기 MVP 범위 원문 ↗

## ② 계획·작업 지침 마련

**개발 계획**

계획·MVP 범위 문서로
구현 범위와 완료 조건 정리

**AI 작업 지침**

AGENTS · Codex 초기 지침

계획·초기 지침 커밋 ↗

## ③ 구현 착수

**플러그인 골격 구현**

플러그인 설정·빌드 구성
Editor 모듈 생성

골격 구현 커밋 ↗

**개발 중 보완** | 기능별 계획·검증·문서 갱신 절차 추가

기능 작업 기준 ↗

### 사용자 작업

구현 전 계획·AGENTS 설정을 먼저 요청
기능별 계획과 검증 가능한 작업 단위 제시

### Codex 작업

계획·초기 지침 문서 작성과 규칙 반영
요청한 기준에 따라 기능 작업 수행

# 플러그인 골격에 연결한 탐색·결과 처리 기능

Editor 모듈 골격을 바탕으로 참조 관계 분석과 트리 표시·에셋 이동·CSV 저장을 구현했습니다.
이후 미사용 후보 탐색을 추가하고, 실행한 기능의 결과를 공통 표시·저장 흐름으로 연결했습니다.

## 초기 계획 | 참조 관계를 트리로 조회

**관계 분석**
에셋 선택 · 방향·깊이·필터

→

**선택 에셋에서 관계 탐색**
사용하는 대상과 사용되는 곳 조회

## 이후 확장 | 프로젝트의 미사용 후보 탐색

**미사용 후보 탐색**
선택 없이 실행 · 폴더·종류

→

**프로젝트 콘텐츠에서 후보 탐색**
다른 에셋이 참조하지 않는 대상 조회

**실행한 기능의 결과로 갱신**

| | |
|---|---|
| 관계 분석 실행 | 참조 관계 트리 |
| 후보 탐색 실행 | 미사용 후보 전용 목록 |

실행할 때마다 이전 결과를 새 결과로 교체

**목록 표시**
참조 관계 트리 또는 후보 목록

→

**원본 에셋 위치로 이동**
표시된 결과 항목을 더블클릭

**CSV 저장**
현재 결과와 해당 분석 모드를 저장

### 구현 위치와 근거

**입력·탐색·표시: Slate 위젯**
Asset Registry · STreeView · RefreshTree
관계·후보 탐색과 UI 구현 ↗

**결과·분석 모드 보관**
TreeRootItems · CurrentTreeMode

**필터·CSV 처리: 별도 헬퍼**
CSV Exporter에 현재 결과·모드 전달
CSV 형식과 저장 책임 ↗

# 참조 탐색에서 공유 관계 유지와 순환 탐색 종료

여러 에셋이 같은 대상을 참조하는 관계는 각 경로에 표시해야 합니다.
같은 탐색 경로에서 다시 만난 에셋은 순환으로 표시하고, 그 아래 탐색을 멈추도록 구현했습니다.

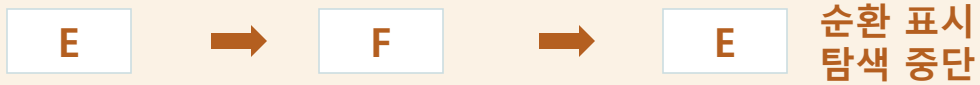
## 공유·순환 관계의 처리 기준

### 공유 | 서로 다른 참조 관계를 모두 표시

B와 C가 같은 D를 참조하므로 두 관계를 유지




### 순환 | 같은 경로에서 지난 에셋을 다시 만남




개발 당시 Blueprint 방어 전후 화면 ↗

## 검증 | 공유·순환 처리 결과

추가 QA · 2026.10.21 · 별도 Data Asset

| 검증 조건 | 기대 동작 | CSV 확인 결과 |
|---|---|---|
| B·C가 같은 D를 참조 | 두 참조 관계 모두 유지 | B→D·C→D 각각 기록 |
| E→F→E 최대 깊이 3 | 되돌아온 E 표시 하위 탐색 중단 | 깊이 2에서 다시 만난 E를 순환으로 표시 최대 깊이 3에 도달 전 종료 |

CSV: IsCircular=true · 순환 E 이후 행 없음

공유 참조 CSV ↗

순환 참조 CSV ↗

## 사용자 작업

말단 관계 보존 판단·순환 감지 상태 요구
방어 전후 비교 제안과 실제 화면 확보

## Codex 작업

CurrentPath 기반 재귀 제어 구현
순환 노드 상태와 표시 코드 추가

추가 QA: Codex의 별도 Data Asset hard reference. 당시 Blueprint·실제 프로젝트 재실행과 크래시·성능 실측은 미수행.

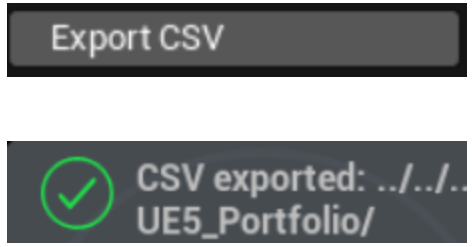
# 화면의 분석 결과를 CSV 파일로 저장

플러그인에서 확인한 참조 관계를 외부에서도 검토할 수 있도록 파일로 남겼습니다.

## ① 실제 분석 화면




## ② CSV 저장




## ③ 실제 CSV 결과

| AssetName | Depth | Mode |
|---|---|---|
| BP_Dummy | 0 | Dependencies |
| M_Dummy | 1 | Dependencies |
| T_Dummy_Color | 2 | Dependencies |

개발 당시 화면·CSV · 오른쪽 표는 일부 열 발췌

## CSV 저장 방식

**저장 대상**
실제 에셋만 저장(안내·그룹 제외)

**처리 구조**
별도 Exporter 호출

**저장 정보**
부모·깊이·모드·순환 여부 저장

원본 화면 ↗ CSV 원본 ↗ 작업 계획 초안 ↗ 구현·검증 기록 ↗

## 사용자 작업

CSV 결과 파일 3개와 화면 5장 제공
직렬화 규칙과 리뷰 범위 검토

## Codex 작업

CSV Exporter와 UI 연결 구현
계획·설계·검증 기록 작성과 빌드 실행

# CSV 개발에 활용한 AI 코드 리뷰

CSV 저장 기능 구현 후 PR #59에서 Codex 자동 리뷰를 활용했습니다.
리뷰에서 지적한 사용 조건을 수정에 반영하고, 빌드 확인과 변경 기록으로 연결했습니다.

## 개발에 적용한 리뷰 흐름

### 01 구현 코드 리뷰

CSV 저장 기능 PR #59
Codex 자동 리뷰 실행

➡

### 02 지적 내용 검토

리뷰가 지적한 사용 조건과
관련 저장 코드 검토

➡

### 03 코드 수정·빌드 확인

지적 사항 수정 반영·빌드 확인
변경 내용과 리뷰 대응 기록

## 적용 사례 | 재분석 없이 방향을 바꿀 때 CSV 모드 불일치

| | |
|---|---|
| **발견한 오류** | 재분석 없이 방향 버튼만 바꾸면, 화면의 결과는 그대로인데<br>CSV의 분석 모드만 바뀌었습니다. |
| **반영한 수정** | 현재 버튼 선택값 대신, 화면의 결과를 생성할 때 사용한<br>분석 모드를 저장합니다. |
| **추가 QA** | 방향 버튼만 변경 → 기존 결과와 CSV 유지<br>다시 분석 → 화면 결과와 CSV 모드 함께 갱신 (2026.10.21) |

자동 리뷰 원문 ↗ 수정 코드 ↗ 추가 QA·원본 CSV ↗ 문서 운영 기준 ↗

### 사용자 작업

리뷰 범위 확인·Live Coding 해제 등 환경 대응
수정 후 커밋·리뷰 대응 절차 진행

### Codex 작업

자동 리뷰로 결함 발견·CSV 모드 수정·빌드
리뷰 대응과 계획·검증 기록 정리

개발 당시: 수정 코드 빌드 확인 · 추가 QA: Codex 실행 / 별도 Data Asset · 과거 전체 빌드 비교 아님

# 그래픽스 학습·구현과 AI 아카이브 자동화

교재·강의 기반 그래픽스 학습·구현과 AI를 활용한 기록 정리·촬영 자동화를 다룹니다.
자동화 과정과 문제 해결 경험을 중심으로, 그래픽스 학습·구현 사례를 함께 소개합니다.

프로젝트 성격

**교재·강의 기반 학습 프로젝트**

그래픽스 예제 학습·구현

그래픽스 구현 환경

**C++ · HLSL · DirectX11**

렌더링·입력·애니메이션·GPU 연산

아카이브 자동화 도구

**Codex · Windows API · FFmpeg**

문서 정리·실행·촬영·형식 검사

## 이 포트폴리오에서 보여주는 경험

### 01 AI 아카이브 자동화

대표 예제로 문서 규격·절차를 검토하고,
Codex로 후속 예제에도 적용했습니다.

산출물·자동화 과정 (41–42페이지) →

### 02 문제 해결과 검증

자동 촬영의 잔존 오류 창 대응과
Assimp 구조 호환 문제를 다룹니다.

문제 해결·검증 (43–48페이지) →

### 03 그래픽스 학습·구현

렌더링·애니메이션·GPU 연산의 학습 내용을
코드와 실행 결과로 설명합니다.

학습·구현 범위 (49–62페이지) →

### 사용자 작업

학습·구현, 정리 목적·규격·촬영 조건 제시
산출물 검토와 오류 대응 방향 보완

### Codex 작업

코드·학습 기록 조사, 문서화·촬영 도구 구현
반복 실행·촬영·검사와 요청에 따른 보완

# 코드·문서·실행 결과 색인

왼쪽의 주제·페이지 링크는 본문으로, 오른쪽의 Git 링크는 관련 원문으로 이동합니다.
그래픽스 학습·구현 01~13은 상세 페이지의 번호와 같습니다.


## AI 아카이브 자동화

자동화 산출물·과정 (41–42페이지) → Git 문서 ↗
문제 해결 01 (43–45페이지) → Git 문서 ↗

## 학습 범위·작업 기록

학습·구현 범위 (49페이지) → Git 저장소 ↗

## 렌더링 기초·표면 표현

01 반사·굴절 (50페이지) → Git 문서 ↗
02 원근 투영·UV 보정 (51페이지) → Git 문서 ↗
03 DirectX11 파이프라인 (52페이지) → Git 문서 ↗
04 Normal Mapping (53페이지) → Git 문서 ↗
05 Height Mapping (54페이지) → Git 문서 ↗
06 Tessellation (55페이지) → Git 문서 ↗

## FBX 구조 호환 문제

문제 해결 02 (46–48페이지) → Git 코드 ↗

## 재질·후처리·입력·애니메이션

07 PBR·IBL (56페이지) → Git 문서 ↗
08 HDR·Bloom (57페이지) → Git 문서 ↗
09 3D 선택·조작 (58페이지) → Git 문서 ↗
10 스켈레탈 애니메이션 (59페이지) → Git 문서 ↗

## GPU 연산·인스턴싱

11 GPU Blur (60페이지) → Git 문서 ↗
12 GPU 연기 (61페이지) → Git 문서 ↗
13 잔디 인스턴싱 (62페이지) → Git 문서 ↗


**링크 안내** 본문 보기: 포트폴리오 내부 이동 / Git 문서·코드: 외부 근거

# 그래픽스 학습·구현과 아카이브 자동화

DirectX11 기반 그래픽스 예제의 학습·구현·실행 기록을 정리한 프로젝트입니다.
분산된 학습 기록을 코드 설명·실행 결과·검증 자료로 구성하고, 반복 정리를 Codex로 자동화했습니다.

## 반복 작업의 규격화

학습 내용과 구현 결과를 설명할 수 있도록,
반복되는 문서·실행·기록 절차를 규격화했습니다.

### 공통 규격을 적용한 README 구성 예시

#### 02_Bloom

원문 발췌 · 중간 생략

**구현 요약**

입력 이미지를 CPU에서 처리한 뒤 dynamic texture로
업로드하고 full-screen quad로 표시한다.

**Build And Run**

| 항목 | 결과 |
|---|---|
| Debug x64 build/run | 성공 |
| Release x64 build/run | 성공 |

**Related Docs**

Topic ↗
Verification ↗
Detailed Demo ↗

공통 README 템플릿 ↗

Bloom README 원문 ↗

## 실제 산출물 · 시각 결과 기록



Bloom 설명·결과 이미지 ↗

**작업 기여** | **사용자 작업:** 정리 목적·기준 제시, 결과 검토 | **Codex 작업:** 코드·기록 조사, 문서 초안 작성, 실행 결과 수집

# AI를 활용한 아카이브 구축 자동화

대표 예제로 문서 규격과 작업 절차를 검토한 뒤,
Codex의 작업 범위를 후속 예제의 문서 정리·실행·촬영·검사로 넓혔습니다.

## 자동화 진행 과정과 작업 기여

### 01 정리 기준 설정

**사용자 작업**

문서·실행 결과의 규격 제시
촬영 방법·조건·제한사항을
정리해 전달

**Codex 작업**

자료 조사·문서 초안 작성
요구에 맞춘 촬영 도구 구현

→

### 02 대표 예제 적용·보완

**사용자 작업**

대표 예제의 결과 검토
필요한 보완 요청

**Codex 작업**

문서·검증 기록·데모 연결
요청에 따른 작업 절차 보완

→

### 03 후속 예제로 확대

**사용자 작업**

진행 상황 관찰
예외 발생 시 방향 조정

**Codex 작업**

후속 예제 문서 정리
실행·촬영·형식 검사

→

### 04 산출물 검수

**사용자 작업**

문서·촬영 결과와
학습 내용 대조
정리 목적에 맞게 보완 요청

**Codex 작업**

보완 의견에 따른 자료 수정
문서와 근거 링크 정리

## 오류 발생 시 대응 방식

**사용자 작업:** 분석 과정 관찰 · 맥락이 어긋나면 방향 조정

**Codex 작업:** 오류 원인 1차 분석

작업 기록 · 대표 예제 적용 ↗

## 촬영 도구의 목적

**목적:** 예제의 시각적 실행 결과를 사진·영상으로 기록

**사용 도구:** Windows API · FFmpeg

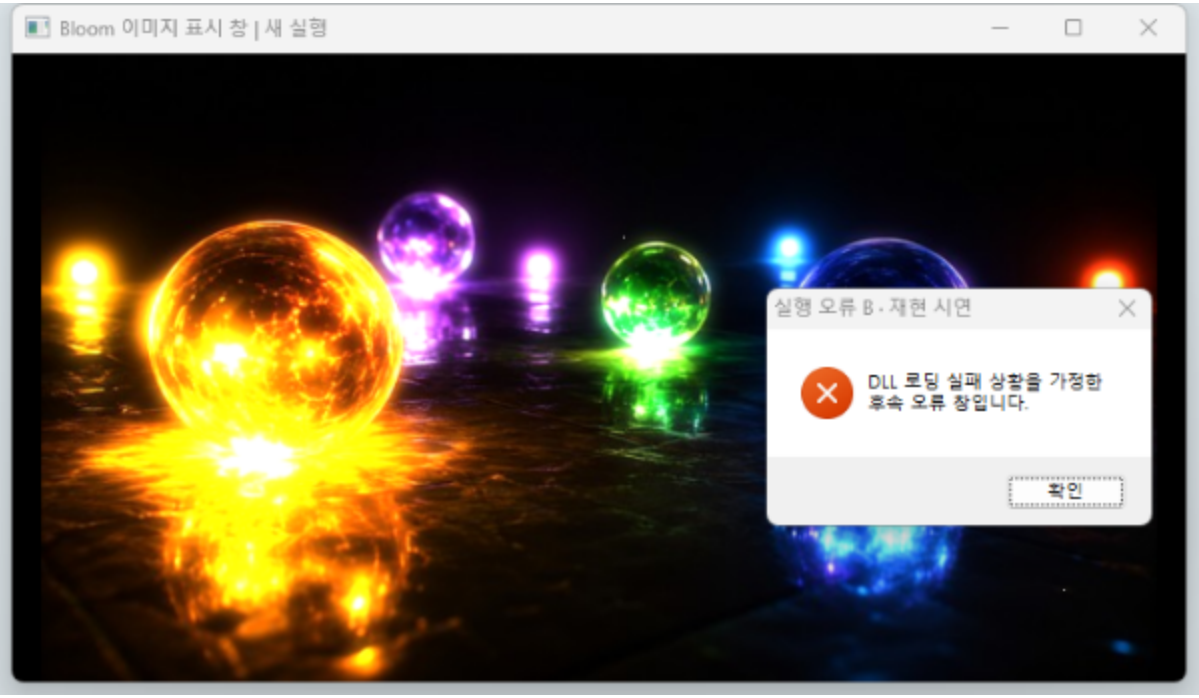
도구 문서 · 실행 화면 촬영 ↗

**자동화 범위** | 예제 검토 · 문서 정리 · 실행 결과 기록 · 형식 검사

# 문제: 실패 상태를 구분하지 못해 자동 촬영 작업 반복

자동 촬영 과정에서 이전 실행의 오류 창이 남아, 새 실행 결과를 판단하는 데 혼선이 생겼습니다.
오류 창 하나를 닫은 뒤 정리 완료를 확인하지 않고 다음 작업을 진행한 것이 원인이었습니다.

## 오류 상황 재현 · 이전 오류 창이 남은 상태




## 작업 흐름 · 오류 대응 반복

Codex가 잔존 오류를 새 실행 문제와 혼동해,
분석·수정·재실행을 반복했습니다.

## 관찰 · 이전 오류가 새 실행에 잔존

오류 A를 닫자 후속 오류 B가 나타났지만,
이를 남긴 채 새 실행을 진행했습니다.

## 원인 · 정리 완료 판단 누락

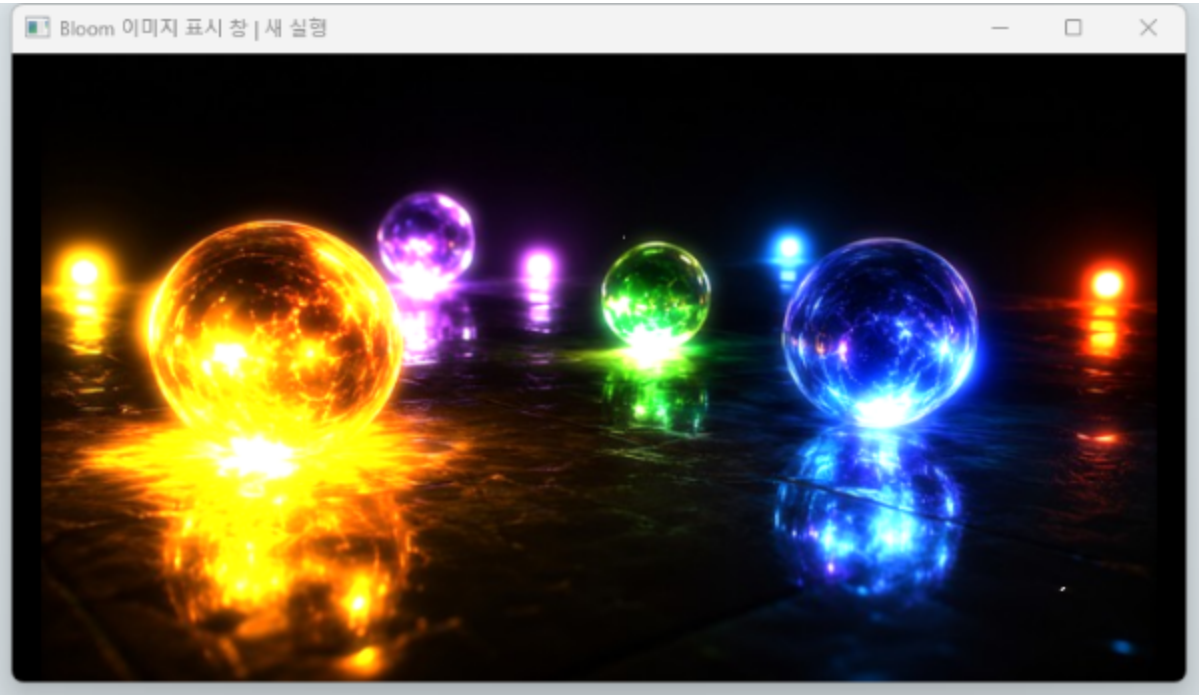
오류 창 하나를 닫은 뒤
후속 창이 남았는지 확인하지 않고
다음 작업으로 진행했습니다.

**작업 기여** | **사용자 작업:** 오류 대응 과정 관찰·방향 제시 | **Codex 작업:** 당시 오류 분석·수정·재실행 시도

# 해결: 오류 정리 결과를 판별하도록 자동화 도구 보완

오류 창을 닫은 뒤 후속 창과 잔존 여부를 재확인하도록 절차를 보완했습니다.
정리 완료와 실패를 반환 코드로 구분해, 후속 작업에서 결과를 판단할 기준을 마련했습니다.

## 재현 시연 · 정리 확인 후 진행하는 절차




## 개선 방향 · 정리 결과 확인 후 다음 작업

도구가 반환한 성공·실패 코드를 확인한 뒤
다음 작업을 결정하도록 제안했습니다.
성공 확인 후 Clean/Rebuild하는 절차입니다.

## 구현 내용 · 반복 정리와 마지막 확인

정리 단계에서는 후속 창을 확인하며 종료를 반복합니다.
같은 창이 남으면 실패 코드를 반환합니다.
정리 후 안정 대기에서 오류가 다시 감지되면,
재정리하지 않고 실패 코드를 반환합니다.

## 검증 범위 · 오류 정리·결과 판별 코드

오류 정리·결과 판별 코드를 검사했습니다.
사진은 절차 설명용 재현 시연입니다.
DLL 복구·Clean/Rebuild·촬영 재개는 미검증입니다.

**작업 기여** | **사용자 작업:** 잔존 오류 재확인 절차 제안 | **Codex 작업:** 반복 정리·재검사·결과 반환 구현

# 검증: 테스트 창으로 오류 정리와 실패 감지 확인

보완된 오류 창 정리 코드를 추가 수정 없이, 테스트 창으로 5개 조건을 각각 3회 실행했습니다.
종료 코드·잔존 오류·일반 창 유지 여부를 확인해, 오류 창 정리 성공과 잔존 오류 감지를 구분했습니다.

## 검증 범위 · 재현용 테스트 창 · 5개 조건 × 3회 · 2026.10.19

| 테스트 조건 | 기대 동작 | 확인한 결과 | 기대 동작 충족 |
|---|---|---|---|
| 오류 없는 정상 상태 | 창을 닫지 않고 정리 절차 종료 | 종료 코드 0 · 일반 창 1개 유지 | 3/3 충족 |
| A 종료 후 B 표시 | B까지 정리 후 성공 코드 반환 | 종료 코드 0 · 오류 잔존 0개 | 3/3 충족 |
| 안정 대기 중 B 표시 | 남은 B 감지 · 성공 판정 안 함 | 종료 코드 3 · B 잔존 1개 감지 | 3/3 충족 |
| 종료 요청을 거부하는 창 | 반복 중단 · 실패 코드 반환 | 종료 코드 3 · 같은 창 잔존 감지 | 3/3 충족 |
| 오류 창과 일반 창 동시 표시 | 오류만 정리 · 일반 창 유지 | 종료 코드 0 · 오류 0 / 일반 창 1 | 3/3 충족 |

실패 감지 조건의 충족은 잔존 오류를 보고했다는 뜻이며, 오류 창 정리 성공을 의미하지 않습니다.

**확인한 한계**

지연 오류는 감지 후 실패 종료하며 자동 재정리하지 않습니다. 실제 DLL 복구·재촬영은 미검증입니다.
위 15회와 별도인 회귀 테스트: 완료 대기 한도 5초의 원본 실패 → 한도만 15초로 늘린 사본 통과.
실패 원인은 확정하지 못했습니다.

# 문제: FBX 임포트 구조 불일치로 캐릭터 형태 붕괴

캐릭터가 한곳에 뭉쳐 정상적인 형태로 표시되지 않았습니다.
임포트된 본 계층과 애니메이션 채널을 확인해, 기존 로더가 기대한 구조와의 차이를 추적했습니다.

## 캐릭터 형태 붕괴 · Pivot 보존 ON



Assimp 6.0.2 · 동일 조건 비교 2026.10 촬영

## 관찰한 현상 · 몸이 한곳에 뭉침

머리·몸·다리가 겹치며 캐릭터의 형태가
무너지고, 보행 자세가 제대로 표현되지 않았습니다.

## 구조 확인 · Pivot 보존으로 달라진 이름

로더가 찾는 본 이름

**mixamorig:Spine**

임포트된 채널 이름 · 실제 로그 발췌

**mixamorig:Spine_$AssimpFbx$_Rotation**

## 원인 · 기존 로더의 채널 매핑 실패

본 이름과 일치하지 않는 채널이 모두 본 인덱스 0에
연결돼, 여러 채널의 변환이 같은 본에 덮어써졌습니다.

보행 클립: 보조 노드 144개 / 미일치 채널 48개

임포트 구조·채널 매핑 로그 ↗

**작업 기여** | **사용자 작업:** 당시 문제 인지·문제 제시 | **Codex 작업:** 이번 동일 조건 재현·구조 로그 수집

# 해결: Pivot 보존 설정을 로더의 처리 구조에 맞춤

기존 로더가 처리하는 본 계층과 채널 이름에 맞도록 Pivot 보존 옵션을 껐습니다.
모델과 애니메이션을 읽는 경로에 같은 설정을 적용했습니다.

## 캐릭터 형태 복원 · Pivot 보존 OFF



앞 페이지와 동일 프레임·배율 2026.10 촬영

## 변경 코드 · 동일한 임포트 설정 적용

ModelLoader의 Load / LoadAnimation에서
ReadFile 호출 전에 동일 옵션을 적용했습니다.

```
importer.SetPropertyBool(
    AI_CONFIG_IMPORT_FBX_PRESERVE_PIVOTS,
    false);
```

## 변경 의미 · 로더가 처리할 수 있는 구조

Pivot 변환을 보조 노드로 분리해 보존하는 대신,
기존 로더가 처리하는 본·채널 구조로 읽었습니다.
임포트 설정을 로더의 처리 조건에 맞춘 변경입니다.

변경 근거: 2026.07 · 커밋 7c061ee
최초 분석·코드 작성자는 기록으로 미확인.

당시 변경·확인 기록 ↗ 당시 실행 확인 원문 ↗

**작업 기여** | **사용자 작업:** 당시 수정 후 Debug/Release 확인 | **Codex 작업:** 이번 옵션 전후 실행·계측

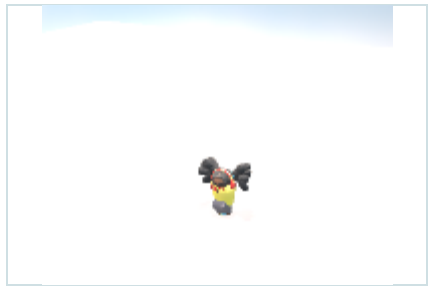
# 검증: 채널 매칭과 보행 정상화 확인

동일한 자산과 Assimp 버전에서 Pivot 보존 옵션을 비교했습니다.
5개 클립의 임포트 구조를 검사하고, 보행 클립의 실제 재생 결과를 확인했습니다.

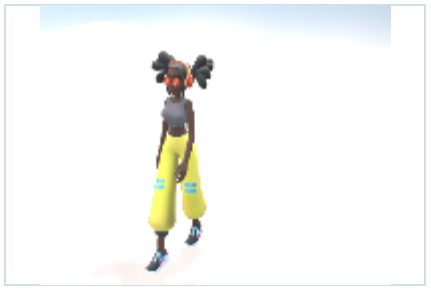
## 옵션 변경 전후 · 구조 검사와 실제 보행 재생

| 검증 항목 | 확인한 결과 · ON → OFF | 검증 범위 |
|---|---|---|
| 임포트 구조와 이름 매칭 | 보조 노드 144 → 0개 / 미일치 채널 48 → 0개 | 5개 클립 각각 |
| 미등록 채널의 잘못된 연결 | 본 인덱스 0으로 잘못 연결한 횟수: 240 → 0회 | 5개 클립 합계 |
| 실제 애니메이션 재생 | 캐릭터 형태와 보행 자세 복원 | 보행 클립 · 조건별 240프레임 |

**Pivot 보존 ON**



**Pivot 보존 OFF**



## 확인 범위와 한계

구조 검사는 5개 클립, 실제 재생 비교는 보행 클립에 한정했습니다.
다른 클립 전체 재생·장시간 실행·임의 자산은 미검증입니다.

### 비교 조건 · 2026.10 · 별도 재현 사본

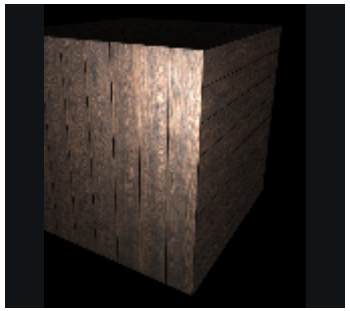
별도 재현 사본 양쪽에 Codex의 초기 자세 보완을 동일 적용하고,
Pivot 옵션만 바꿔 비교했습니다.
초기 자세 보완은 원본에 반영하지 않았습니다.

**작업 기여** | **사용자 작업:** 당시 수정 결과 직접 실행 확인 | **Codex 작업:** 이번 동일 조건 비교·로그·영상 수집

# 대표 그래픽스 학습·구현 범위

교재·강의 기반 예제를 학습하며 구현·실행한 결과를 정리한 아카이브입니다.
학습 예제와 이후 Codex의 조사·문서화·실행 자료 정리를 구분합니다.

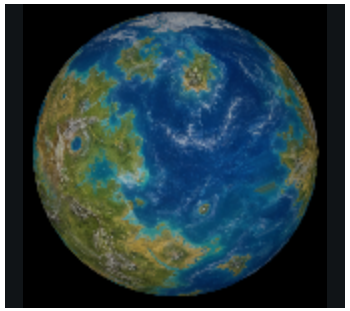
## DirectX11 렌더링 기초



리소스·셰이더 연결
텍스처 기반 화면 출력

예제 문서·결과 이미지 ↗

## 절차적 메시 생성



정점·인덱스 구성
법선·UV 생성

예제 문서·결과 이미지 ↗

## 재질·조명



PBR·IBL
그림자 표현

예제 문서·결과 이미지 ↗

## 사용자 상호작용



화면 좌표 기반 피킹
쿼터니언 회전·이동

예제 문서·결과 이미지 ↗

## 스켈레탈 애니메이션




본 계층 변환·스키닝
애니메이션 클립 재생

예제 문서·결과 이미지 ↗

## GPU 연산



Compute Shader
속도·밀도 갱신

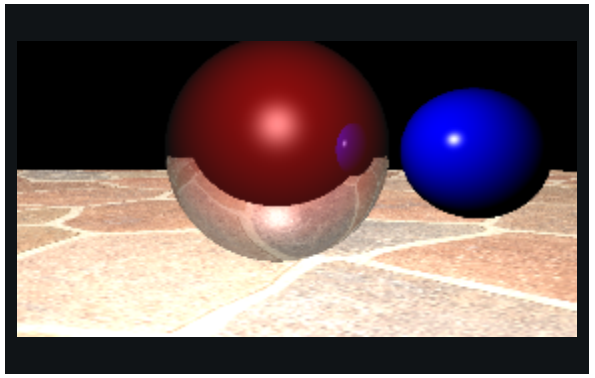
예제 문서·결과 이미지 ↗

**공통 범위** | **사용자 작업:** 학습·정리 기준 제시·결과 검토 | **Codex 작업:** 코드·학습 기록 조사, 문서화·실행 자료 정리

# CPU Ray Tracing의 반사·굴절

광선과 물체의 교차점을 찾고, 반사·굴절 방향의 광선을 재귀적으로 추적했습니다.
CPU에서 계산한 픽셀 데이터를 DirectX11 텍스처에 복사해 화면에 표시했습니다.

## 반사 예제



## 굴절 예제



서로 다른 예제 장면의 결과이며, 동일 장면의 적용 전후 비교는 아닙니다.

## 핵심 원리 · 반사·굴절 광선

교차점의 법선과 굴절률로 후속 광선의 방향을 구하고,
그 광선이 도달한 표면의 색을 계산합니다.

## 구현 방식 · 최근접 교차점과 재귀 추적

최근접 교차점을 선택하고 재귀 깊이를 제한합니다.
시작점에 작은 오프셋을 더해 자기 교차를 줄입니다.

## 결과 해석 · 반사상과 굴절 표현

각 예제에서 반사상과 굴절된 배경 표현을 확인합니다.

## 반사 광선의 재귀 추적 · C++

원문 발췌

반사 광선을 추적한 색에 반사율을 곱해 누적합니다.

```
color += traceRay(reflectedRay, depth - 1)
    * hit.obj->reflection;
```

반사 문서·결과 이미지 ↗

반사 구현 코드 ↗

## 굴절 광선의 방향 계산 · C++

원문 발췌

광선 방향·법선·굴절률 비로 굴절 방향을 구합니다.

```
const vec3 refractedDir =
    glm::refract(ray.dir, normal, eta);
```

굴절 문서·결과 이미지 ↗

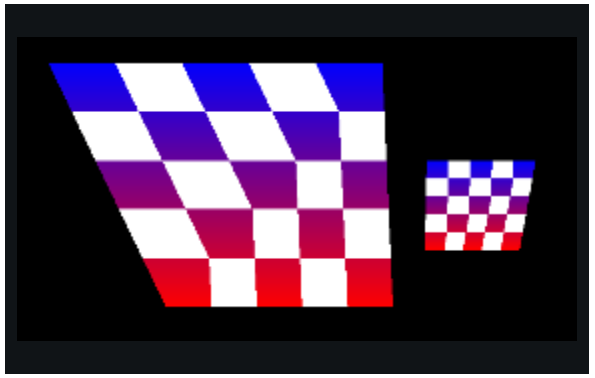
굴절 구현 코드 ↗

**학습·구현 범위** | 교차점 판정, 반사·굴절 광선의 재귀 추적과 픽셀 버퍼 출력

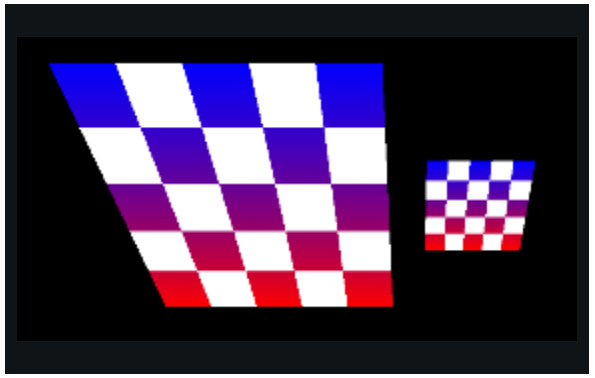
# 원근 투영과 UV 보간 보정

원근 투영 이후의 화면 공간 가중치에 깊이를 반영해 UV를 보간했습니다.
같은 위치·투영 조건에서 보정 전후 체크 패턴의 변화를 비교한 기존 자료입니다.

## UV 보정 전



## UV 보정 후



화면 거리 1.000 · 왼쪽 Z 0.309 · 오른쪽 Z 2.490 고정

## 핵심 원리 · 원근 투영 뒤의 보간

화면 공간의 보간 비율은 원래 공간의 비율과 다릅니다.
깊이를 반영해야 텍스처의 원근감을 유지할 수 있습니다.

## 구현 방식 · 깊이로 나눈 가중치

화면 가중치를 눈 기준 깊이로 나누고 다시 정규화합니다.
이 예제는 CPU에서 UV를 보간하고 픽셀을 계산합니다.

## 결과 해석 · 체크 패턴의 변화

같은 위치·투영 조건에서 보정 전후의 패턴을 비교합니다.
DirectX11은 CPU가 계산한 픽셀 버퍼를 화면에 표시합니다.

## 원근 보정 가중치 계산 · C++

원문 발췌

화면 가중치를 깊이로 나눈 뒤 전체 합으로 정규화합니다. 다른 정점도 같은 방식으로 처리합니다.

```
const float ratio0 = w0 / z0;
const float ratioTotal = ratio0 + ratio1 + ratio2;
w0 = ratio0 / ratioTotal;
```

예제 문서·결과 이미지 ↗

깊이 보정 코드 ↗

**학습·구현 범위** | 깊이를 반영한 가중치 정규화와 원근 보정 UV 보간

# DirectX11의 데이터·셰이더 연결

정점·인덱스·텍스처와 상수 버퍼를 연결해, 형상과 조명을 화면에 표시했습니다.
예제의 실제 바인딩 코드와 DrawIndexed 호출을 기준으로 데이터 흐름을 정리했습니다.

## 텍스처·조명 적용 결과



Texturing LightingSelf 실행 결과

**01 입력 데이터**
정점·인덱스·텍스처·상수

↓

**02 셰이더 처리**
Vertex Shader / Pixel Shader

↓

**03 렌더 타깃**
색상 출력 / 깊이 검사

## 핵심 원리 · 입력과 셰이더의 연결

정점·인덱스는 형상을, 텍스처·상수 버퍼는
변환과 재질·조명 계산에 필요한 값을 제공합니다.

## 구현 방식 · 바인딩과 DrawIndexed

정점·인덱스 버퍼와 셰이더 입력을 연결합니다.
렌더 타깃·깊이 버퍼를 설정하고 DrawIndexed로 출력합니다.

## 결과 해석 · 형상·재질·조명의 결합

실행 화면에서 텍스처와 조명이 적용된 형상을 확인합니다.
개별 입력의 효과를 분리한 비교 실험은 아닙니다.

원문 발췌

## 텍스처·인덱스 버퍼 연결과 그리기 · C++

픽셀 셰이더에 텍스처를, 입력 단계에 인덱스 버퍼를 연결한 뒤 삼각형을 그립니다.

```
m_context->PSSetShaderResources(0, 1, pixelResources_0);
m_context->IASetIndexBuffer(m_indexBuffer.Get(), DXGI_FORMAT_R16_UINT, 0);
m_context->DrawIndexed(m_indexCount, 0, 0);
```

예제 문서·결과 이미지 ↗

리소스 연결·Draw 호출 코드 ↗

**학습·구현 범위** | 정점·인덱스·텍스처·상수 버퍼 연결과 DrawIndexed 호출

# Normal Mapping의 법선 변환

Normal Map의 방향을 표면의 TBN 기저로 변환해 조명 계산에 사용했습니다.
정점 위치를 바꾸지 않고, 표면의 밝기 변화로 요철을 표현하는 예제입니다.

## Normal Mapping · Off



## Normal Mapping · On



같은 영역·배율로 발췌 · 조명 반응의 변화 비교

## 핵심 원리 · 법선으로 표현하는 요철

Normal Map의 방향을 조명 계산에 사용해
정점 위치를 바꾸지 않고 표면의 요철을 표현합니다.

## 구현 방식 · 방향 복원과 TBN 변환

RGB 값을 -1~1 범위로 복원하고 TBN 기저로 변환합니다.
월드 공간의 법선을 정규화해 조명 계산에 사용합니다.

## 결과 해석 · 조명 반응과 형상의 차이

같은 영역에서 표면의 밝기·무늬 변화를 비교합니다.
형상·외곽선이나 별도의 자기 그림자를 만들지는 않습니다.

## 법선 복원과 TBN 변환 · HLSL

원문 발췌

텍스처의 방향 값을 복원하고 월드 공간으로 변환한 뒤 정규화합니다.

```
normalTex = 2.0 * normalTex - 1.0;
float3x3 TBN = float3x3(T, B, N);
normalWorld = normalize(mul(normalTex, TBN));
```

예제 문서·결과 이미지 ↗

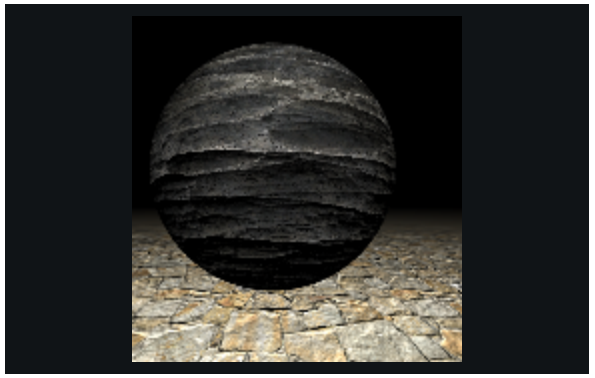
법선·TBN 변환 코드 ↗

**학습·구현 범위** | 법선 맵 방향 복원과 TBN 변환을 통한 조명 계산

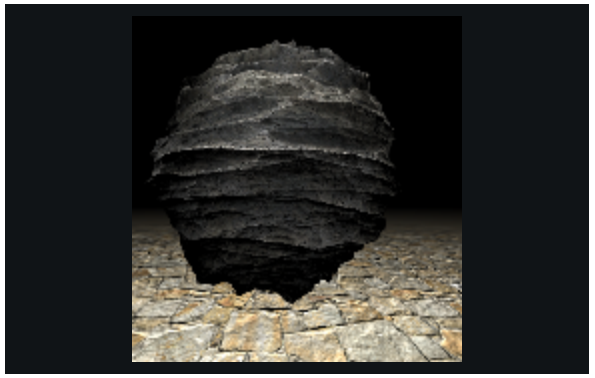
# Height Mapping의 정점 변위

Vertex Shader에서 높이 텍스처를 읽고 법선 방향으로 정점을 이동했습니다.
Normal Mapping과 달리, 실제 표면 위치와 외곽선이 달라지는 예제입니다.

## Height Mapping · Off



## Height Mapping · On



같은 영역·배율로 발췌 · Height Scale 0.1 적용

## 핵심 원리 · 높이 값에 따른 정점 변위

높이 텍스처를 이용해 정점을 법선 방향으로 이동합니다.
Normal Mapping과 달리 실제 표면 위치가 바뀝니다.

## 구현 방식 · 높이 복원과 변위 적용

Vertex Shader에서 높이를 -1~1로 복원합니다.
법선·높이·Height Scale을 곱해 정점 위치에 더합니다.

## 결과 해석 · 외곽선과 정점 밀도

Scale 0.1 적용 전후의 표면·외곽선을 비교합니다.
정점 밀도에 영향을 받으며, 변위 후 법선은 갱신하지 않습니다.

## 높이 값에 따른 정점 이동 · HLSL

원문 발췌

높이를 -1~1로 복원하고 법선 방향의 변위를 정점 위치에 더합니다.

```
height = 2.0 * height - 1.0;
pos += float4(output.normalWorld * height * heightScale, 0.0);
```

예제 문서·결과 이미지 ↗

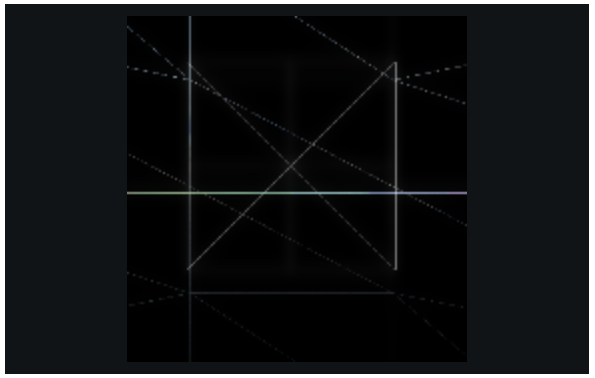
정점 변위 코드 ↗

**학습·구현 범위** | 높이 텍스처를 이용한 법선 방향 정점 변위

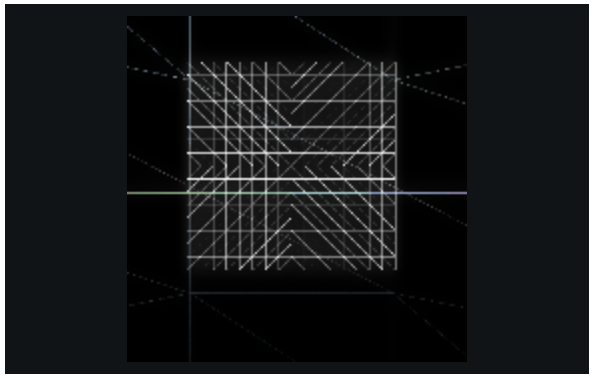
# Tessellation의 분할 밀도 제어

네 개의 Control Point로 구성한 패치를 Hull Shader의 분할 계수에 따라 세분화했습니다.
같은 카메라에서 Manual 계수만 바꾼 기존 실행 결과로 삼각형 밀도를 비교했습니다.

## Manual · Factor 2



## Manual · Factor 16



동일 카메라 · Manual 분할 계수 2 / 16

## 핵심 원리 · 패치의 세분화

분할 계수를 높이면 패치의 정점·삼각형이 늘어납니다.
Hull·Domain Shader가 분할과 정점 위치 계산을 나눕니다.

## 구현 방식 · 분할 계수와 정점 위치

Hull Shader가 외곽·내부 분할 계수를 설정하고,
Domain Shader는 패치 내부 좌표로 정점 위치를 구합니다.

## 결과 해석 · Factor 2와 16의 차이

같은 카메라에서 Manual 계수에 따른 삼각형 밀도를 봅니다.
거리 적응형 분할은 이 비교에 포함하지 않습니다.

## 패치 분할 계수 전달 · HLSL

원문 발췌

외곽·내부 분할 계수를 패치 상수에 기록합니다. 정점 위치 계산은 Domain Shader에서 수행합니다.

```
pt.edges[0] = edges.x;
pt.inside[0] = inside.x;
pt.inside[1] = inside.y;
```

예제 문서·결과 이미지 ↗

분할 계수 코드 ↗

**학습·구현 범위** | Hull Shader 분할 계수와 Domain Shader의 정점 위치 계산

# PBR·IBL의 재질 입력과 조명 계산

재질 맵과 환경광 텍스처를 셰이더 입력으로 연결한 PBR·IBL 학습 예제입니다.
각 입력의 역할을 실제 코드와 여러 맵을 함께 적용한 실행 결과로 정리했습니다.

## PBR·IBL 종합 적용 결과



여러 재질 맵과 Height Scale 0.015를 함께 적용한 결과

**재질 입력**
Albedo·Metallic
Roughness
Normal·AO

**환경광 입력**
Irradiance
Specular 맵
LUT

**조명 결과**
표면 방향·시선·거칠기 반영

## 핵심 원리 · 재질과 환경광

재질·표면 방향·시선 방향에 따라 조명 반응을 구합니다.
환경 맵은 주변에서 들어오는 빛의 입력으로 사용합니다.

## 구현 방식 · 재질 맵과 IBL 샘플링

재질 맵과 Irradiance·Specular 환경 맵, LUT를 연결합니다.
Roughness에 따라 반사 환경 맵의 Mip Level을 선택합니다.

## 결과 해석 · 여러 입력의 종합 결과

재질·환경광 맵과 Height Scale 0.015를 함께 적용했습니다.
각 맵의 효과를 하나씩 분리한 비교는 아닙니다.

## 확산·반사 환경광 샘플링 · HLSL

원문 발췌

법선 방향의 확산광과 반사 방향의 환경광을 읽고, 거칠기로 반사 샘플의 Mip Level을 정합니다.

```
float3 irradiance = irradianceIBLTex.Sample(linearSampler, normalWorld).rgb;
float3 specularIrradiance = specularIBLTex.SampleLevel(linearSampler,
    reflect(-pixelToEye, normalWorld), roughness * 10.0).rgb;
```

예제 문서·결과 이미지 ↗

IBL 샘플링 코드 ↗

**학습·구현 범위** | 재질 맵 입력과 환경 맵 샘플링을 사용하는 PBR·IBL 조명 계산

# HDR·Bloom의 후처리 흐름

부동소수점 장면 버퍼에서 Bloom 이미지를 만들고, 원본과 합성했습니다.
현재 활성 코드의 가중 혼합·노출·감마 처리를 기준으로 후처리 흐름을 설명합니다.

### Bloom Strength · 0



### Bloom Strength · 0.7



같은 영역·배율로 발췌 · 원본과 Bloom의 가중 혼합

### 핵심 원리 · 빛 번짐과 합성

해상도를 줄인 이미지를 다시 확대해 밝은 영역을 퍼뜨립니다.
이를 원본 장면과 혼합해 Bloom 효과를 만듭니다.

### 구현 방식 · 가중 혼합과 표시 변환

원본과 Bloom을 (1-strength), strength 비율로 섞습니다.
이후 노출·0~1 제한·감마 처리를 적용합니다.

### 결과 해석 · 강도에 따른 변화

같은 영역에서 Strength 0과 0.7의 빛 번짐을 비교합니다.
현재 혼합식은 강도가 커질수록 원본의 비중도 줄어듭니다.

### 원본·Bloom 혼합과 표시 변환 · HLSL

원문 발췌

두 이미지의 비중을 strength로 조절하고, 선택된 LinearToneMapping 경로로 결과를 변환합니다.

```
float3 combined = (1.0 - strength) * color0 + strength * color1;
combined = LinearToneMapping(combined);
```

예제 문서·결과 이미지 ↗

Bloom 합성·톤 매핑 코드 ↗

**학습·구현 범위** | Bloom 이미지 생성·가중 합성과 노출·감마 처리

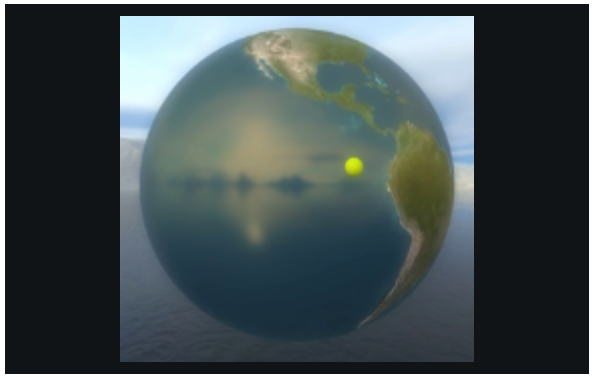
# 화면 입력과 3D 선택·조작

화면 좌표를 선택 광선으로 바꾸고, 구 표면의 드래그 입력으로 회전을 적용했습니다.
Virtual Trackball 예제의 선택 지점과 회전 누적 과정을 기존 자료로 설명합니다.

## 드래그 전



## 드래그 중



기존 시연의 초기·드래그 상태 · 같은 영역·배율로 발췌 · 노란 점은 선택 지점

## 핵심 원리 · 화면 입력을 3D 회전으로

화면 좌표로 선택 광선을 만들고 구 표면의 지점을 구합니다.
직전·현재 선택 지점의 방향 차이를 회전량으로 사용합니다.

## 구현 방식 · 선택 판정과 회전 누적

교차를 검사하고 두 벡터 사이의 쿼터니언을 구합니다.
모델에 회전을 합성하고 현재 벡터를 다음 기준으로 둡니다.

## 결과 해석 · 드래그에 따른 자세 변화

드래그 전·중 상태에서 모델의 회전을 확인합니다.
노란 점은 선택 지점이며, 연속 동작은 조작 영상에 있습니다.

## 선택 판정과 회전량 계산 · C++

원문 발췌

선택 광선의 교차를 검사하고 두 방향 사이 회전을 구합니다.

```
m_selected = curRay.Intersects(m_mainBoundingSphere, dist);
q = Quaternion::FromToRotation(prevVector, currentVector);
```

예제 문서·결과 이미지 ↗

## 모델 행렬에 회전 합성 · C++

원문 발췌

구한 쿼터니언을 회전 행렬로 바꿔 기존 변환에 합성합니다.

```
m_mainSphere.UpdateModelWorld(m_mainSphere.m_modelWorldRow *
    Matrix::CreateFromQuaternion(q) *
    Matrix::CreateTranslation(translation));
```

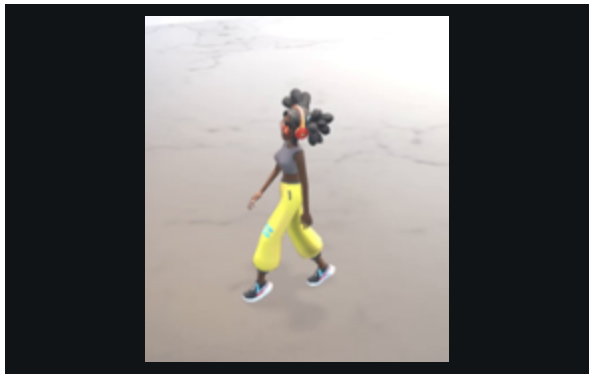
선택·회전 처리 코드 ↗

**학습·구현 범위** | 화면 좌표 기반 피킹과 쿼터니언 회전 누적

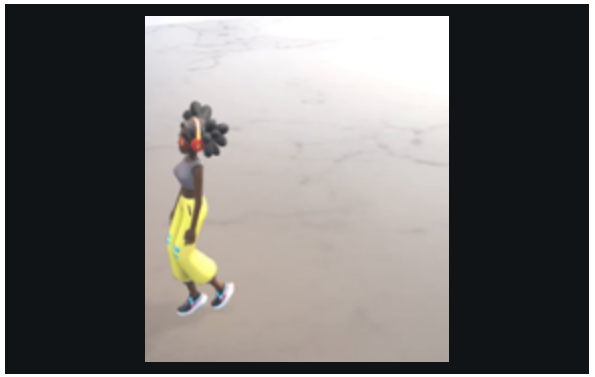
# 스켈레탈 애니메이션의 변환 흐름

클립의 본 변환을 계층에 따라 갱신하고, GPU에서 가중합으로 정점을 변형했습니다.
기존 애니메이션 시연의 두 시점을 같은 배율로 발췌했습니다.

### 기존 시연 · 0.800초



### 기존 시연 · 4.300초



동일 클립 재생의 0.800초·4.300초 장면 · 같은 배율로 발췌

### 핵심 원리 · 본 변환의 가중합

각 정점에 연결된 본의 변환을 가중치만큼 합산합니다.
여러 본의 영향으로 메시가 애니메이션 자세로 변형됩니다.

### 구현 방식 · CPU 갱신과 GPU 스키닝

클립·프레임의 본 계층 변환을 계산해 GPU로 전달합니다.
셰이더는 본 인덱스·가중치로 위치·법선·접선을 변환합니다.

### 결과 해석 · 동일 클립의 두 시점

0.800초와 4.300초의 자세 차이로 재생 결과를 확인합니다.
FBX 호환 문제의 원인과 해결은 46–48페이지에서 다룹니다.

### 본 변환에 따른 정점 스키닝 · HLSL

원문 발췌

각 본으로 변환한 위치·법선·접선을 해당 가중치만큼 더합니다. 아래는 반복문 내부의 누적 연산입니다.

```
posModel += weights[i] * mul(float4(input.posModel, 1.0f), boneTransforms[indices[i]]).xyz;
normalModel += weights[i] * mul(input.normalModel, (float3x3)boneTransforms[indices[i]]);
tangentModel += weights[i] * mul(input.tangentModel, (float3x3)boneTransforms[indices[i]]);
```

예제 문서·결과 이미지 ↗

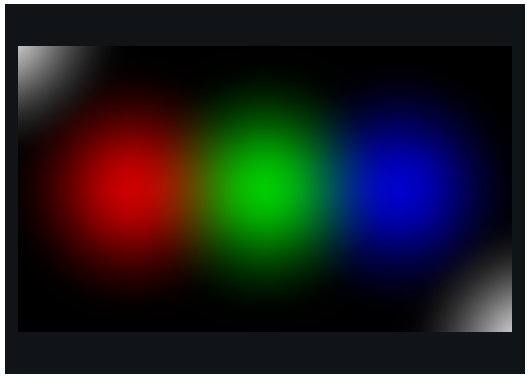
정점 스키닝 코드 ↗

**학습·구현 범위** | 본 계층 변환 갱신과 본 가중치를 이용한 GPU 스키닝

# GPU Blur의 구현 경로와 리소스 교대

A/B 리소스를 읽기·쓰기 대상으로 교대하며 Blur의 X/Y 방향 계산을 수행했습니다.
현재 활성 Compute Shader 경로의 데이터 전달과 Dispatch 순서를 설명합니다.

## 기존 Blur 실행 결과



**A · 입력 리소스**

① X 방향 계산 · A → B

**B · X 방향 계산 결과**

② Y 방향 계산 · B → A

**A · Y 방향 계산 결과**

기존 실행 결과 · 구현 흐름을 설명하며 성능 비교를 제시하지 않습니다.

## 핵심 원리 · 입력과 출력의 교대

한 단계의 출력이 다음 단계의 입력이 되도록
A/B 리소스를 번갈아 읽고 쓰는 방식입니다.

## 구현 방식 · SRV·UAV와 X/Y 패스

X 방향은 A 읽기·B 쓰기, Y 방향은 B 읽기·A 쓰기입니다.
패스 사이에 바인딩을 해제하고 각 커널을 Dispatch합니다.

## 결과 해석 · Blur 결과와 검증 범위

활성 Compute Shader 경로로 생성한 Blur 결과입니다.
바인딩 해제는 GPU 계산 완료를 기다리는 동작이 아닙니다.

## ① X 패스의 입력·출력 연결 · C++

원문 발췌

A를 읽는 SRV와 B에 쓰는 UAV를 연결합니다.

```
m_context->CSSetShaderResources(0, 1, m_srvA.GetAddressOf());
m_context->CSSetUnorderedAccessViews(
    0, 1, m_uavB.GetAddressOf(), NULL);
```

예제 문서·결과 이미지 ↗

## ② Y 패스의 입력·출력 연결 · C++

원문 발췌

앞 단계 결과 B를 읽고 A에 쓰도록 역할을 바꿉니다.

```
m_context->CSSetShaderResources(0, 1, m_srvB.GetAddressOf());
m_context->CSSetUnorderedAccessViews(
    0, 1, m_uavA.GetAddressOf(), NULL);
```

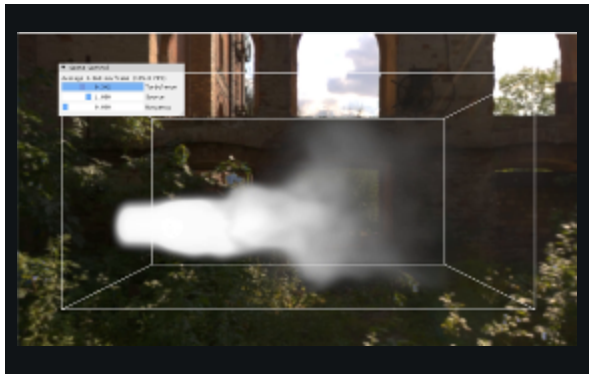
A/B 리소스 교대 코드 ↗

**학습·구현 범위** | SRV/UAV 바인딩 해제와 A/B 리소스 교대를 통한 X/Y Blur

# GPU 연기의 시뮬레이션과 볼륨 렌더링

Compute Shader로 3차원 속도·밀도 필드를 갱신하고, 밀도 텍스처를 렌더링했습니다.
기존 영상의 연기 형태 변화와 속도를 이용한 밀도 이송 연산을 정리했습니다.

### 4.025초



### 22.808초



동일 시연의 4.025초·22.808초 장면 · 같은 영역·배율로 발췌

### 핵심 원리 · 속도에 따른 밀도 이동

3차원 속도·밀도 필드를 갱신해 연기의 이동을 표현합니다.
속도를 거슬러 찾은 이전 위치의 밀도를 현재 셀에 반영합니다.

### 구현 방식 · 필드 갱신과 볼륨 표시

Source 주입·발산·압력 계산·속도 보정 뒤 밀도를 이송합니다.
Ray Marching으로 밀도를 누적해 화면에 표시합니다.

### 결과 해석 · 시간에 따른 연기 형태

동일 시연의 4.025초·22.808초에서 연기 형태를 비교합니다.
시각 결과를 보여주며, 물리 정확도를 측정한 자료는 아닙니다.

### 속도를 거슬러 밀도 샘플링 · HLSL

원문 발췌

속도·시간 간격으로 이전 위치를 역추적하고, 그 위치의 밀도를 읽어 현재 값을 갱신합니다.

```
float3 vel = velocityTemp[dtID.xyz].xyz * dx;
float3 uvwBack = uvw - (vel * dt * upScale);
density[dtID.xyz] = densityTemp.SampleLevel(linearWrapSS, uvwBack, 0.0) * 0.999;
```

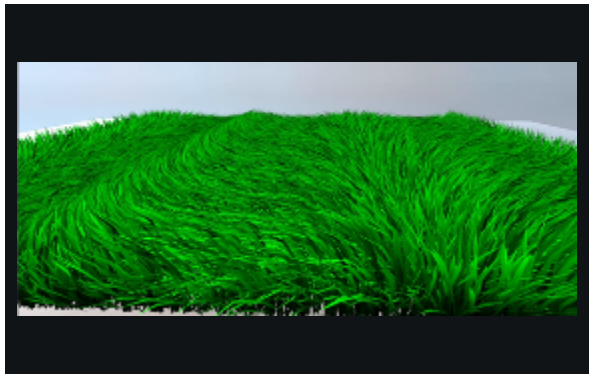
예제 문서·결과 이미지 ↗ 시뮬레이션 단계 코드 ↗ 밀도 이송 셰이더 ↗

**학습·구현 범위** | 속도·밀도 필드 갱신과 Ray Marching 기반 볼륨 렌더링

# 잔디 인스턴싱과 셰이더 변형

하나의 잔디 형상과 개별 배치 데이터를 분리해 인스턴스 렌더링을 구성했습니다.
Vertex Shader에서 위치·시간·바람 세기로 잔디의 움직임을 표현했습니다.

### 기존 시연 · 1.190초



### 기존 시연 · 6.743초



동일 시연의 시간에 따른 잔디 방향 변화

### 핵심 원리 · 형상 공유와 인스턴스 출력

잔디 형상은 공유하고 위치·회전·크기는 개별 데이터로 둡니다.
인스턴스 데이터로 같은 형상을 여러 위치에 출력합니다.

### 구현 방식 · 배치와 바람 변형

두 버퍼를 연결하고 DrawIndexedInstanced를 호출합니다.
셰이더가 위치·시간·바람에 따라 정점과 법선을 회전시킵니다.

### 결과 해석 · 잔디 움직임과 구현 범위

100,000개를 구성하고 시간에 따른 잔디 방향 변화를 봅니다.
충돌·GPU Culling·LOD는 구현 범위에 포함하지 않습니다.

### 인스턴스 단위로 형상 출력 · C++

원문 발췌

같은 인덱스 형상을 인스턴스 수만큼 출력합니다.

```
context->DrawIndexedInstanced(
    m_indexCount, m_instanceCount, 0, 0, 0);
```

### 바람 회전을 정점에 적용 · HLSL

원문 발췌

바람 회전을 행렬로 만들고 월드 정점 위치에 적용합니다.

```
float4 q = rotate_angle_axis(input.windStrength, axis);
mWind = quaternion_to_matrix(q);
output.posWorld = mul(float4(output.posWorld, 1.0f), mWind).xyz;
```

예제 문서·결과 이미지 ↗ 인스턴스 출력 코드 ↗ 바람 변형 셰이더 ↗

**학습·구현 범위** | 인스턴스 데이터 기반 반복 출력과 시간·바람에 따른 정점 변형